

Putting the OPTI in Round Optimal IA-MPC in the Plain Model

Yashvanth Kondi¹, Divya Ravi², Jure Sternad³, and Sophia Yakoubov³

¹ Silence Laboratories, Singapore; yash@ykondi.net

² University of Amsterdam, Netherlands; d.ravi@uva.nl

³ Aarhus University, Denmark; {jssternad, sophia.yakoubov}@cs.au.dk

Abstract. We advance the study of best-possible security guarantees of two-round secure multiparty computation in the plain model.

While Goel et al. (TCC '21) showed the infeasibility of Identifiable Abort in two rounds in the plain model with a standard honest majority (i.e. $t < n/2$), the effect of relaxing the setting to that of Selective Identifiable Abort, or an honest supermajority (i.e. $t < n/3$), has not been established. In this work, we close this gap.

In the honest supermajority case, we show a positive result: two rounds *are* sufficient to achieve identifiable abort, and thus also selective identifiable abort. Notably, ours is the first two-round general purpose MPC construction in the plain model tolerating more than one corruption that achieves a guarantee stronger than unanimous abort.

We obtain our construction via a new compiler that lifts any protocol with unanimous abort, to one that achieves identifiable abort. A core building block for our compiler is *Oblivious Public Transfer with Intermediaries* (OPTI), a new primitive that we introduce which may be of independent interest.

When a third or more of the parties might be corrupt, we show a negative result: three rounds are necessary to achieve Selective Identifiable Abort with straight-line simulation.

Table of Contents

PUTTING THE OPTI IN ROUND OPTIMAL IA-MPC IN THE PLAIN	
MODEL	1
<i>Yashvanth Kondi, Divya Ravi, Jure Sternad, and Sophia Yakoubov</i>	
1 Introduction.....	3
1.1 Related Work	4
1.2 Our Results	4
1.3 Technical Overview: Lower Bound	5
1.4 Technical Overview: Upper Bound	5
2 Impossibility of Two Round MPC with SIA for $t \geq \frac{n}{3}$	7
3 Oblivious Public Transfer with Intermediaries (OPTI)	9
3.1 OPTI Definitions	10
3.2 OPTI Protocol	14
3.3 Proof of Security	19
4 Definition of MPC with Identifiable Abort	21
5 Construction of Two-Round MPC with IA for $t < \frac{n}{3}$	23
5.1 Proof of Security	27
A Preliminaries	33
A.1 Public Key Encryption	33
A.2 Commitment Scheme	34
A.3 Secret Sharing	35
A.4 Multi-string Non-Interactive Zero-knowledge Proofs.	36
A.5 Garbling Scheme	39
B Hybrids for Proof of Theorem 3	41

1 Introduction

Secure Multiparty Computation (MPC, [CDvdG88,GMW87,Yao86]) allows multiple parties, each with a private input, to compute a joint function of their private inputs without revealing anything about their inputs apart from the function output. MPC has been an active area of research for over 50 years now. A key goal has been making MPC protocols more *secure* and more *efficient*, while reducing the infrastructure it relies on (*setup*).

Security. The security of MPC protocols is measured in two crucial ways:

Adversary’s Corruption Power: How many of the n parties can an adversary control (that is, what is the corruption threshold t)?

Security Guarantee: Which outcomes can an adversary induce?

The most-studied security guarantees are as follows, roughly from weakest to strongest:

Selective Abort (SA): The adversary may deprive some of the honest parties of output.

Unanimous Abort (UA): The adversary may deprive the honest parties of output, but if it does, the honest parties all agree that the computation aborted. (Any protocol that achieves UA also achieves SA.)

Selective Identifiable Abort (SIA) [DRSY23a]: The adversary may deprive some of the honest parties of output. However, the honest parties who do not receive output are able to identify at least one corrupt party. (Any protocol that achieves SIA also achieves SA.)

Identifiable Abort (IA) [AL10]: The adversary may deprive the honest parties of output, but if it does, the honest parties all agree that the computation aborted, and they are able to unanimously identify at least one corrupt party. (Any protocol that achieves IA also achieves UA and SIA.)

Fairness (FAIR): The adversary may deprive the honest parties of output, but if it does, the adversary does not learn the output either. (Any protocol that achieves fairness also achieves UA.)

Guaranteed Output Delivery (GOD): The adversary may not deprive the honest parties of output. (Any protocol that achieves GOD also achieves all other guarantees.)

Efficiency. There are many axes along which the efficiency of MPC protocols is measured; among the most-studied are communication complexity and round complexity. MPC protocols are known to require at least two rounds of communication [HLP11]. In this paper, we focus on protocols that run in exactly two rounds.

Setup. Some MPC protocols rely on a common reference string (CRS); others require a public key infrastructure (PKI), where all participants know one-anothers’ public keys ahead of time; yet others use an even more involved form of setup, where participants hold *correlated randomness*. In this paper, we focus on protocols that require *no* setup. This is known as the *plain model*.

In this paper, we advance the study of what security is achievable in two rounds in the plain model, for various values of the corruption threshold t .

1.1 Related Work

Round-optimal MPC with computational security has been an active area of research under various models, including with setup [GGHR14,MW16,GS18,BL18] and without setup [KO04,IKKP15,BHP17,ACJ17,CCG⁺20]. Round optimality with information-theoretic security was investigated in [AKP20,ABT21,AKP23]. Below, we outline the works most relevant to our setting, namely plain model and computational security.

When an adversary might control a majority of participants ($t \geq \frac{n}{2}$), four rounds are necessary for any security guarantee [GMPP16]. We thus focus on the honest majority setting.

When a majority of participants is guaranteed to be honest ($t < \frac{n}{2}$), UA (and thus SA) is achievable in two rounds [ACGJ18], while IA is not [GJPR21], and neither are fairness or GOD [PR18,GLS15].

In the case of what is known as an honest “supermajority” ($t < \frac{n}{3}$), fairness and GOD are still unachievable in two rounds as long as $t > 1$ [GIKR02], SA [IKP10] and UA is still achievable (the construction of [ACGJ18] naturally still works if fewer parties are corrupt).

There are some crucial gaps in the landscape, which we resolve in this paper.

1.2 Our Results

Prior to this work, it was unknown whether SIA is achievable in two rounds in the plain model if the adversary controls $\frac{n}{3} \leq t < \frac{n}{2}$ participants. We show that if straight-line simulation is required, it is *not* (Theorem 1), implying that three rounds are necessary for SIA in this setting⁴.

Prior to this work, it was also unknown whether IA — or SIA — are achievable in two rounds in the plain model if the adversary controls $t < \frac{n}{3}$ participants. We show that they *are* (Theorem 3). Our construction is the first two-round general purpose MPC construction in the plain model tolerating more than one corruption to achieve any guarantee stronger than unanimous abort.

Table 1 summarizes our results (positive results highlighted in green, negative in orange), as well as the rest of the landscape described above.

⁴Straight-line simulation is the standard in the presence of an honest majority, and is required for universal composability. However, it remains an open problem to determine whether SIA is achievable in two rounds in the plain model for $\frac{n}{3} \leq t < \frac{n}{2}$ if straight-line simulation is not required.

Threshold	Selective Abort (SA)	Unanimous Abort (UA)	Selective Identifiable Abort (SIA)	Identifiable Abort (IA)	Fairness (FAIR)	Guaranteed Output Delivery (GOD)
$\frac{n}{3} \leq t < \frac{n}{2}$	✓ ←	✓[ACGJ18] [‡] ↓	✗(Thm. 1)	✗[GJPR21]	✗[PR18, GLS15]	→ ✗
$t < \frac{n}{3}$	✓ ←	✓	✓ ←	✓ (Thm. 3)	✗ for $t > 1$ [GIKR02]	→ ✗ for $t > 1$

Table 1: A map of the feasibility and impossibility of two-round MPC with different guarantees, organized by thresholds.

1.3 Technical Overview: Lower Bound

In Section 2, we prove the impossibility of realizing SIA in two rounds in the plain model if $t \geq \frac{n}{3}$. In our proof we assume — towards contradiction — a two-round three-party SIA protocol in this setting. We call our three parties $\mathcal{P}_1$, $\mathcal{P}_2$ and $\mathcal{P}_3$; since $t \leq \frac{n}{3}$, an adversary may corrupt at most one of them.

From the point of an honest $\mathcal{P}_3$, a corrupt $\mathcal{P}_2$ who does not send first-round private messages to $\mathcal{P}_1$ looks just like a corrupt $\mathcal{P}_1$ claiming not to have received them. Since $\mathcal{P}_3$ can thus not blame either of its peers, in order to achieve SIA, $\mathcal{P}_3$ must produce the correct output.

If a corrupt $\mathcal{P}_2$ misbehaves only by withholding its first-round private messages to $\mathcal{P}_1$, we argue that a straight-line simulator must be able to extract corrupt $\mathcal{P}_2$ ’s “correct” input using only its first-round broadcast and private message to $\mathcal{P}_3$. We reach our contradiction by observing that a semi-honest $\mathcal{P}_3$ could then use such a straight-line simulator to extract $\mathcal{P}_2$ ’s input directly, violating privacy.

We extend this lower bound to protocols among $n > 3$ parties by applying the standard player-emulation argument.

1.4 Technical Overview: Upper Bound

In Section 5, we construct a protocol that computes any function f with identifiable abort in two rounds in the plain model if $t < \frac{n}{3}$.

Oblivious Public Transfer with Intermediaries (OPTI). As a building block, we define a new primitive which we call Oblivious Public Transfer with Intermediaries (OPTI). OPTI is a twist on Oblivious Transfer (OT), where a sender supplies two inputs x_0 and x_1 , and a receiver supplies a choice bit b and receives x_b as output. The security properties of OT are that the sender should not learn b , and the receiver should not learn x_{1-b} .

Like OT, OPTI has a sender and a receiver where the sender supplies x_0 and x_1 , and the receiver supplies b . Unlike OT, OPTI is run among n parties, where in addition to the sender and receiver, there are $n - 2$ parties who provide no input (we call these *intermediaries*). Also unlike OT, OPTI has *public output*, meaning

that everyone learns x_b . This means that if $x_0 \neq x_1$, the sender naturally learns b ; however, OPTI requires that if neither the sender nor receiver are corrupt, the adversary should not learn b (privacy), and if the sender is not corrupt, the adversary should not learn x_{1-b} (sender-privacy).

In addition to privacy and sender-privacy, OPTI demands *identifiability*, meaning that if honest parties don't get the right output they should be able to blame a corrupt sender or receiver.

Using OPTI To Build Secure Computation with Identifiable Abort. We use a two-round OPTI protocol that achieves security in the presence of fewer than $\frac{n}{3}$ corruptions to build a two-round secure computation protocol Π_{IA} which achieves security with identifiable abort in the same setting.

We start with a two-round secure computation protocol Π_{UA} that achieves unanimous abort in this setting [ACGJ18]. In order to turn Π_{UA} identifiable, we add zero knowledge proofs of correct behavior. Because we are in the plain model, instead of using non-interactive zero-knowledge proofs (NIZKs), which require a common reference string (CRS), we use *multi-string* NIZKs (Appendix A.4), where each participant generates their own CRS. Parties create their proofs with respect to the entire set of CRSs, and the proofs are secure as long as a majority of the CRSs were generated honestly. Our participants publish their CRSs in the first round, and publish the proofs of correct behavior in the second round.

If Π_{UA} only used public channels, we would be done; however, Π_{UA} relies on private messages between participants, and it is impossible — without further modification — for a party to publicly prove that it sent another party a correct private message, in the absence of public encryption keys. Since we operate in the plain model, parties can generate and publish encryption keys in the first round (thus eliminating the need for private channels in the second round), but this doesn't help with first-round private communication.

To address this, in Π_{IA} , instead of running Π_{UA} directly, each party garbles (Appendix A.3) the function that computes its second-round messages in Π_{UA} from its state and incoming first-round messages. For each pair of input wire labels corresponding to a bit of incoming first-round message, the garbler plays the role of sender in OPTI; the sender of the corresponding first-round message bit plays the role of receiver, and everyone learns the label. The garbler broadcasts the circuit, together with labels for input wires corresponding to its local state. The circuits are then publicly evaluated to obtain all parties' second-round messages, which in turn can be used to learn the output.

The multi-string NIZKs of correct behavior are modified to prove correct garbling, and those NIZKs — together with the identifiability of OPTI — are enough to guarantee that if honest parties do not learn the correct output, they will identify a corrupt party.

Realizing OPTI. We build two-round OPTI in the plain model by having the sender and receiver both Shamir share their inputs with degree- t in the first round. Leveraging the homomorphic properties of Shamir secret sharing, in the second round, $n \geq 3t + 1$ parties can locally compute, and broadcast, degree- $2t$

shares of $(1 - b)x_0 + bx_1 = x_b$. To enable identification, the sender and receiver commit to their shares, distribute the openings along with the shares, and use multi-string NIZKs to prove the validity of the committed sharings.

One issue that comes up is that some parties may (claim to) not have received valid shares and openings from the sender or the receiver over private channels in the first round⁶. If more than t parties claim not to have heard from someone, that is enough to identify that party as corrupt. However, it could be that t parties claim they didn't hear from the sender, and a different t parties claim they didn't hear from the receiver, which doesn't let us identify either. Since those $2t$ parties cannot compute their shares of x_b , this leaves us with only $n - 2t = t + 1 < 2t$ degree- $2t$ shares — not enough to reconstruct x_b .

To solve this, we use the fact that if the sender is corrupt, the adversary is allowed to learn all of the inputs. If a party does not get valid shares of x_0 and x_1 from the sender in the first round, she facilitates the public recovery of all of her shares. Towards this public recovery, in the first round, each party $\mathcal{P}$ broadcasts a public encryption key. In the second round, the sender encrypts $\mathcal{P}$'s shares to her key, and broadcasts those ciphertexts (along with proofs that they contain the right shares).

Since this enables decryption only after the second round, this comes too late for $\mathcal{P}$ to locally compute and broadcast her share of x_b ; however, if she has not heard from the sender in the first round (and thus knows the sender is corrupt), she can broadcast her secret decryption key in the second round — before even receiving the ciphertext — along with her share of b , enabling the public decryption of her shares of x_0 and x_1 , and the public computation of her degree- $2t$ share of x_b .

2 Impossibility of Two Round MPC with SIA for $t \geq \frac{n}{3}$

Goel *et al* [GJPR21] show that identifiable abort (IA) is unachievable in two rounds for $n \leq 3t$ in the plain model. We observe that their impossibility result can be extended to selective identifiable abort (SIA) with minor modifications. One such minor — but crucial — modification is to the function being computed.

For simplicity, we let $n = 3$ and $t = 1$. Consider the following oblivious transfer function:

$$f(0, b, (c_0, c_1)) = (c_b, c_b, 0),$$

where 0 is a default value which is distinct from an abort (where abort is denoted $\perp$).

Theorem 1. *There does not exist a two-round three-party protocol for f that achieves SIA against a single corruption with straight-line simulation in the plain model.*

⁶If parties knew one another's public encryption keys, they could provably send private messages. However, since we operate in the plain model, the parties do not have known public keys in the first round.

Proof. Towards a contradiction, assume that there exists a two-round three-party protocol Π that computes f with SIA security and straight-line simulation in the plain model.

Assume without loss of generality that the second-round communication involves communication only over broadcast channels. (Any private communication in the second round can be emulated via exchanging private random masks in the first round and broadcasting the masked messages in the second round.)

Consider the following scenarios in an execution of the protocol Π :

Scenario 1C: $\mathcal{P}_1$ is corrupt and behaves honestly, apart from pretending that it did not receive the first-round point-to-point message from $\mathcal{P}_2$.

Scenario 2C: $\mathcal{P}_2$ is corrupt and behaves honestly, apart from dropping the first round point-to-point message to $\mathcal{P}_1$.

Scenario 1SH: $\mathcal{P}_1$ is (passively) corrupt and behaves honestly.

Scenario 3SH: $\mathcal{P}_3$ is (passively) corrupt and behaves honestly.

Lemma 1. *The output of $\mathcal{P}_3$ in Scenario 2C must not be $\perp$ (with all but negligible probability).*

Proof (of Lemma 1). For contradiction, assume that the output of $\mathcal{P}_3$ is $\perp$ in Scenario 2C. Then, based on the SIA guarantee, $\mathcal{P}_3$ must identify $\mathcal{P}_2$ as the corrupt party. However, since the view of $\mathcal{P}_3$ is identically distributed in Scenarios 2C and 1C, $\mathcal{P}_3$ must output $(\perp, \mathcal{P}_2)$ identifying $\mathcal{P}_2$ as the cheater in Scenario 1C as well (with overwhelming probability). This yields a contradiction, because $\mathcal{P}_2$ is honest in Scenario 1C.

Now, consider the simulator $\mathcal{S}_2$ for Scenario 2C. We prove two statements about this simulator: it must be able to extract directly after the first round (Lemma 2), and it must extract the correct input (Lemma 3).

Lemma 2. *The simulator $\mathcal{S}_2$ for scenario 2C must be able to extract $\mathcal{P}_2$'s input directly after the first round.*

Proof (of Lemma 2). Corrupt $\mathcal{P}_2$ has to be able to learn the output c_b , since honest $\mathcal{P}_2$ does in Scenario 1C, and the view of corrupt $\mathcal{P}_2$ in Scenario 2C subsumes the view of honest $\mathcal{P}_2$ in Scenario 1C.

The simulator thus needs to provide corrupt $\mathcal{P}_2$ with enough information to learn the output. If it cannot extract $\mathcal{P}_2$'s input until after the second round, it will not have enough time to program this output into honest party messages to $\mathcal{P}_2$.

Lemma 3. *The simulator $\mathcal{S}_2$ for scenario 2C must extract b (the input used).*

Proof (of Lemma 3). We show this by contradiction. Imagine that $\mathcal{S}_2$ extracts $1 - b$ with non-negligible probability. Then, in the ideal execution, with non-negligible probability honest $\mathcal{P}_1$ outputs c_{1-b} .

Since the outputs of honest parties in the ideal execution must be distributed indistinguishably from those in the real execution, it must be that also in the real execution, with non-negligible probability $\mathcal{P}_1$ outputs c_{1-b} .

Next, consider Scenario 1SH. Let $\tau_1 = \{\{\text{BC}_i\}_{i \in [3]}, \{\text{msg}_{j \rightarrow 1}^1\}_{j \in \{2,3\}}\}$ denote the message transcript that the semi-honest $\mathcal{P}_1$ receives, comprising of all broadcast messages, and the first-round point-to-point messages from $\mathcal{P}_2$ and $\mathcal{P}_3$ to $\mathcal{P}_1$. Since everyone (including $\mathcal{P}_1$) behaves honestly, it must hold that $\mathcal{P}_1$ obtains c_b as the output.

The semi-honest $\mathcal{P}_1$ can execute the following attack: take a truncated transcript $\tau'_1 = \{\{\text{BC}_i\}_{i \in [3]}, \perp, \text{msg}_{3 \rightarrow 1}^1\}$, where the message $\text{msg}_{2 \rightarrow 1}^1$ is dropped. Since τ'_1 is identically distributed to the transcript received by honest $\mathcal{P}_1$ in Scenario 2C, the semi-honest $\mathcal{P}_1$ can compute the output of Scenario 2C. Recall that in this case, we concluded that $\mathcal{P}_1$ outputs c_{1-b} with non-negligible probability. This yields a contradiction, as this demonstrates an attack by semi-honest $\mathcal{P}_1$ where with non-negligible probability it can learn both c_0 and c_1 , which is not allowed as per ideal realization of f .

We conclude that the simulator $\mathcal{S}_2$ (corresponding to corrupt $\mathcal{P}_2$ in Scenario 2C) must be able to extract b using only $\mathcal{P}_2$'s first-round broadcast message (say, BC_2^1) and $\mathcal{P}_2$'s first-round point-to-point message to $\mathcal{P}_3$ (say, $\text{msg}_{2 \rightarrow 3}^1$). Note that both these are a part of $\mathcal{P}_3$'s view in Π .

We now consider Scenario 3SH, where the adversary corrupts $\mathcal{P}_3$ passively. Let $(\text{BC}_2^1, \text{msg}_{2 \rightarrow 3}^1)$ denote $\mathcal{P}_2$'s broadcast messages and first-round point-to-point message to $\mathcal{P}_3$. The semi-honest $\mathcal{P}_3$ can run the straight-line simulator $\mathcal{S}_2$ above on $(\text{BC}_2^1, \text{msg}_{2 \rightarrow 3}^1)$ to extract b — which contradicts privacy, as $\mathcal{P}_3$ must not learn b as per the ideal realization of f .

This concludes the proof.

Corollary 1. *Theorem 1 implies that there exist n -party functionalities f such that there does not exist a two-round n -party protocol for f that achieves SIA security for $t \geq \frac{n}{3}$ in the plain model.*

Corollary 1 follows by the standard logic: otherwise, a three-party protocol could be built by having each of the three parties execute the code of a third of the parties in the n -party protocol.

3 Oblivious Public Transfer with Intermediaries (OPTI)

In this section, we introduce a new building block called Oblivious Public Transfer with Intermediaries (OPTI). As in standard Oblivious Transfer (OT), a sender S holds a pair of messages (x_0, x_1) and a receiver R holds a choice bit b , with the transferred value being x_b . The key difference is that, in OPTI, the output x_b is public and delivered to all participating parties, referred to as intermediaries.

Although the public output allows S to infer R 's choice whenever $x_0 \neq x_1$, OPTI ensures *privacy* in the sense that no information about b is revealed to the adversary as long as both S and R are honest. In addition, if the transfer fails, meaning the parties do not obtain the value x_b , this failure can only be caused by corruption of either S or R , and *identifiability* guarantees that all parties are

able to identify the corrupt party. Finally, as in standard OT, OPTI guarantees *sender-privacy*: if the sender is honest, the adversary learns nothing about x_{1-b} .

Looking ahead, OPTI enables the verifiable transfer of private messages (denoted as b above) from a sender to a receiver in our IA protocol, ensuring that the transfer either succeeds or results in unanimous cheater identification.

Below, we give a formal description of OPTI in Section 3.1 and present a 2-round construction realizing OPTI in Section 3.2.

3.1 OPTI Definitions

The OPTI protocol involves a sender S , a receiver R , and $n-2$ additional parties. We do not assign fixed indices for S and R , as different parties may assume these roles. The algorithm executed by party $\mathcal{P}_k$ in Round r ($r \in [2]$) is denoted OPTI_r^k , while the output computation algorithm is denoted by $\text{OPTI}_{\text{output}}$ (which is identical for all parties).

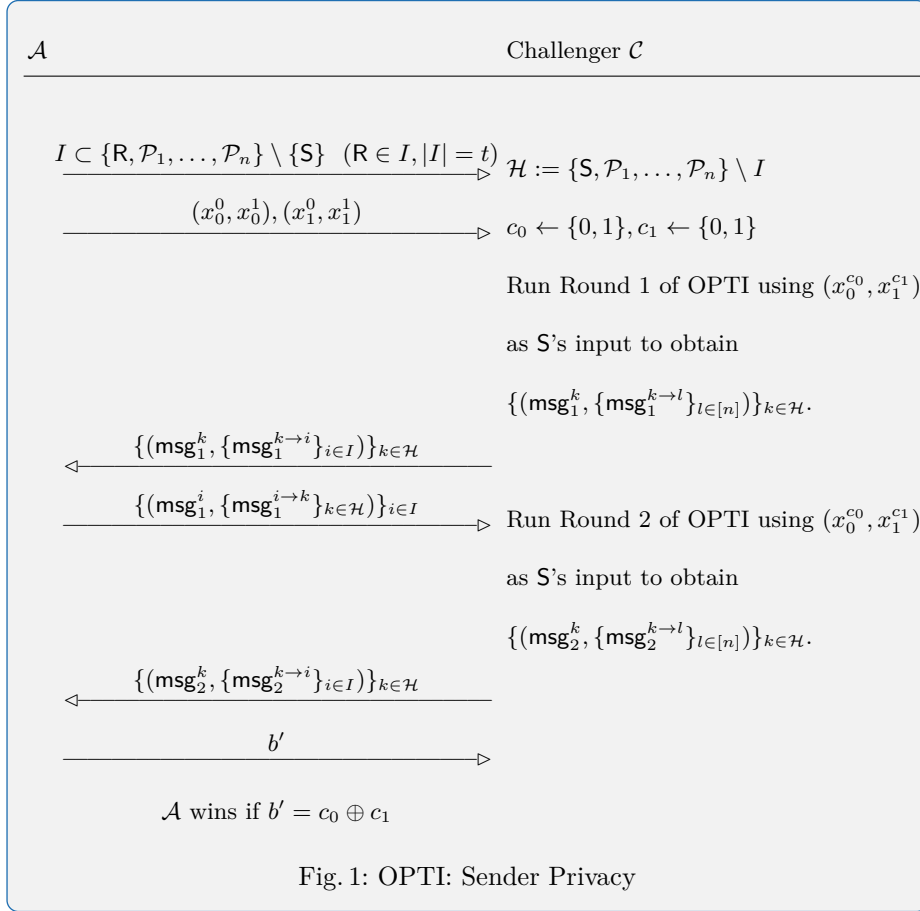
Definition 1 (Oblivious Public Transfer with Intermediaries – OPTI).

A two-round 1-out-of-2 oblivious public transfer with intermediaries OPTI = $((\text{OPTI}_1^R, \text{OPTI}_2^R), (\text{OPTI}_1^S, \text{OPTI}_2^S), ((\text{OPTI}_1^k, \text{OPTI}_2^k)_{k \in [n]}), \text{OPTI}_{\text{output}})$ is an interactive protocol among n parties, where one party plays the role of the receiver R , one party plays the role of the sender S , and every party $k \in [n]$ (including R and S) plays the role of an intermediary party $\mathcal{P}_k$. The receiver R has input $b \in \{0, 1\}$, the sender S has input (x_0, x_1) , while intermediary parties $\mathcal{P}_k$ for $k \in [n] \setminus \{R, S\}$ have no input. At the end of the protocol, every party $\mathcal{P}_k$ outputs $\text{OPTI}_{\text{output}} = x_b$. We require that an OPTI protocol satisfies sender privacy (definition 2), privacy (definition 3), and identifiability (definition 4).

Next, we present game-based definitions ⁷ that formalize the security guarantees of OPTI, namely *sender-privacy*, *privacy* and *identifiability*. In the games below, we use msg_r^k and $\text{msg}_r^{k \rightarrow l}$ to denote $\mathcal{P}_k$'s r th round (where $r \in [2]$) broadcast and point-to-point communication to $\mathcal{P}_l$ respectively.

Definition 2 (OPTI: Sender Privacy). *An OPTI scheme is sender-private if for security parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, it holds that $\Pr[\mathcal{A} \text{ wins}] \leq \frac{1}{2} + \text{negl}(\lambda)$ in the experiment in Figure 1.*

⁷We choose game-based definitions to highlight the exact security properties that we need.



Definition 3 (OPTI: Privacy). An OPTI scheme is private if for security parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, it holds that $\Pr[\mathcal{A} \text{ wins}] \leq \frac{1}{2} + \text{negl}(\lambda)$ in the experiment in Figure 2.

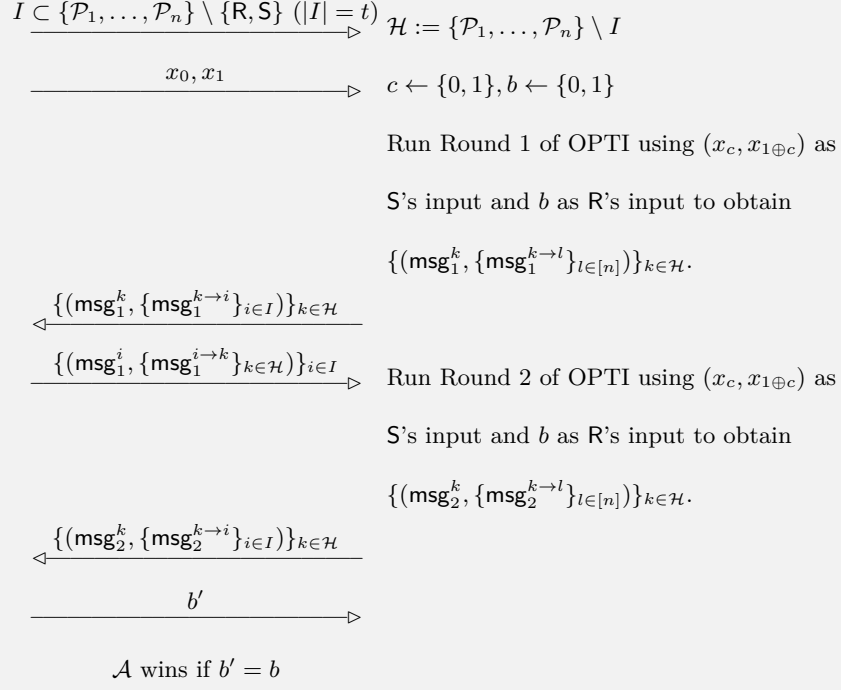
$\mathcal{A}$ Challenger $\mathcal{C}$ 

Fig. 2: OPTI: Privacy

Definition 4 (OPTI: Identifiability). *An OPTI scheme achieves identifiability if for security parameter $\lambda \in \mathbb{N}$, for every PPT adversary $\mathcal{A}$, it holds that $\Pr[\mathcal{A} \text{ wins}] \leq \text{negl}(\lambda)$ in the experiment in Figure 3.*

$\mathcal{A}$	Challenger $\mathcal{C}$
$I \subset \{\mathcal{P}_1, \dots, \mathcal{P}_n\} (I = t)$	$\mathcal{H} := \{\mathcal{P}_1, \dots, \mathcal{P}_n\}.$
a) If $S, R \in \mathcal{H} : b, (x_0, x_1)$	Run Round 1 of OPTI using (x_0, x_1) as S's input and b as R's input to obtain $\{(\text{msg}_1^k, \{\text{msg}_1^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
b) If $S \in \mathcal{H}$ and $R \in I : (x_0, x_1)$	Run Round 1 of OPTI using (x_0, x_1) as S's input to obtain $\{(\text{msg}_1^k, \{\text{msg}_1^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
c) If $S \in I$ and $R \in \mathcal{H} : b$	Run Round 1 of OPTI using b as R's input to obtain $\{(\text{msg}_1^k, \{\text{msg}_1^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
d) If $S, R \in I : \emptyset$	Run Round 1 of OPTI using to obtain $\{(\text{msg}_1^k, \{\text{msg}_1^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
$\{(\text{msg}_1^k, \{\text{msg}_1^{k \rightarrow i}\}_{i \in I})\}_{k \in \mathcal{H}}$	
$\{(\text{msg}_1^i, \{\text{msg}_1^{i \rightarrow k}\}_{k \in \mathcal{H}})\}_{i \in I}$	a) If $S, R \in \mathcal{H} :$ Run Round 2 of OPTI using (x_0, x_1) as S's input and b as R's input to obtain $\{(\text{msg}_2^k, \{\text{msg}_2^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
	b) If $S \in \mathcal{H}$ and $R \in I :$ Run Round 2 of OPTI using (x_0, x_1) as S's input to obtain $\{(\text{msg}_2^k, \{\text{msg}_2^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
	c) If $S \in I$ and $R \in \mathcal{H} :$ Run Round 2 of OPTI using b as R's input to obtain $\{(\text{msg}_2^k, \{\text{msg}_2^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
$\{(\text{msg}_2^k, \{\text{msg}_2^{k \rightarrow i}\}_{i \in I})\}_{k \in \mathcal{H}}$	d) If $S, R \in I :$ Run Round 2 of OPTI to obtain $\{(\text{msg}_2^k, \{\text{msg}_2^{k \rightarrow l}\}_{l \in [n]})\}_{k \in \mathcal{H}}.$
$\{(\text{msg}_2^i, \{\text{msg}_2^{i \rightarrow k}\}_{k \in \mathcal{H}})\}_{i \in I}$	

$\mathcal{A}$ wins any of the following hold:

- $S, R \in \mathcal{H}$ and $\exists \mathcal{P}_k \in \mathcal{H}$ s.t. $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, \text{abort}_R, x\}$ for $x \neq x_b$,
- $S \in \mathcal{H}$ and $R \in I$ and $\exists \mathcal{P}_k \in \mathcal{H}$ s.t. $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, x\}$ for $x \notin \{x_0, x_1\}$,
- $S \in I$ and $R \in \mathcal{H}$ and $\exists \mathcal{P}_k \in \mathcal{H}$ s.t. $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_R\}$,
- $S, R \in I$ and $\exists \mathcal{P}_k \in \mathcal{H}$ s.t. $\text{OPTI}_{\text{output}}^k = \text{abort}$,
- $\exists \mathcal{P}_k, \mathcal{P}_l \in \mathcal{H}$ s.t. $\text{OPTI}_{\text{output}}^k \neq \text{OPTI}_{\text{output}}^l$.

Fig. 3: OPTI: Identifiability

3.2 OPTI Protocol

In this section, we present a two-round construction that realizes OPTI. At a high level, in the first round both the sender S and receiver R threshold-share their inputs— (x_0, x_1) and b respectively—with threshold $(t + 1)$ among the intermediaries. To facilitate verifiable transfer of these shares, we make S and R additionally broadcast commitments of these shares, while sending the corresponding openings privately to the intended recipients.

The key idea is that each intermediary $\mathcal{P}_k$ can now locally compute a share of x_b using its shares of x_0 , x_1 , and b via the relation

$$\langle x_b \rangle_k = (1 - [b]_k) \cdot [x_0]_k + [b]_k \cdot [x_1]_k$$

where $[m]_k$ denotes a threshold- $(t + 1)$ share of m held by party $\mathcal{P}_k$, and $\langle m \rangle_k$ denotes a threshold- $(2t + 1)$ share of m .

Several challenges arise in making this approach work. Firstly, the above computation does not directly yield a fully private threshold- $(2t + 1)$ -sharing of x_b , so we use the standard technique of randomizing it by adding a $(2t + 1)$ -sharing of 0 (generated by S in the first round).

Second and most crucially, reconstructing x_b from these shares requires at least $(2t + 1)$ parties to provide their shares $\langle x_b \rangle_k$, but only parties that successfully received shares (consistent with the relevant public commitments) from both the sender and receiver can do so. Note that when no cheater is identified, we are guaranteed that at least $n - t \geq (2t + 1)$ parties successfully received shares from S , and at least $n - t \geq (2t + 1)$ parties successfully received shares from R . This is because any party complained about by more than t participants can be unanimously identified as corrupt. However, this only guarantees that $n - t - t \geq t + 1$ parties are happy with both S and R . This unfortunately does not suffice for robust reconstruction of a degree- $2t$ shared value, which we require when no cheater is identified.

To address this, we leverage the fact that if S is corrupt, the adversary is allowed to learn all of the inputs: x_0 , x_1 , and b . If a party $\mathcal{P}_k$ doesn't hear from S (but hears from R), it triggers the public release of all its shares: in the second round, it broadcasts its share $[b]_k$, together with a secret key the corresponding public key for which it had published in the previous round. Simultaneously — in the second round — S must broadcast encryptions of $[x_0]_k$ and $[x_1]_k$ to this public key.

These ciphertexts are only decrypted if $\mathcal{P}_k$, unhappy with S , publishes the secret key and $[b]_k$; if it does, this suffices for everyone to compute the share $\langle x_b \rangle_k$ via the relation stated above. Notably, this approach enables us to utilize the shares of every party who is happy with the receiver, which — unless the receiver is identified as corrupt — gives us enough shares to reconstruct x_b .

To summarize, our protocol proceeds as follows:

1. In the first round, both S and R distribute degree- t shares of their inputs. They broadcast commitments to these shares and privately send the corresponding openings to the intended recipients. In addition, all parties publish their public keys.

2. In the second round, parties that received shares from both S and R (denoted by $\mathcal{S}_{S,R}$) compute their degree- $2t$ shares and broadcast them along with proofs of correctness. Moreover, S broadcasts encryptions of the degree- t shares intended for each party $\mathcal{P}_k$ under $\mathcal{P}_k$'s public key. Each $\mathcal{P}_k$ broadcasts her secret key and the share received from R *only if* $\mathcal{P}_k \in \mathcal{S}_R$ (i.e., she received a share from R but not from S in the first round).
3. To compute the output, parties retrieve the degree- t shares corresponding to $\mathcal{P}_k \in \mathcal{S}_R$ (by decrypting the ciphertexts using the secret key broadcast by $\mathcal{P}_k$) and compute the degree- $2t$ shares on behalf of $\mathcal{P}_k$. These are combined with the shares received via broadcast from $\mathcal{S}_{S,R}$ to reconstruct x_b .

Informally, the above protocol design ensures that the protocol successfully computes the output whenever no party is identified as cheating because of the following reason: An adversary attempting to avoid detection can withhold shares from at most t honest parties on behalf of a corrupt S and at most t honest parties on behalf of a corrupt R. The most delicate situation arises when these two sets are disjoint. In that case, up to $2t$ (among $\geq 2t + 1$) honest parties may be missing some shares, preventing them from locally computing their degree- $2t$ shares. However, the adversary gains no real advantage by dropping shares on behalf of S. This is because the protocol design forces the potentially corrupt S (if it wishes to avoid detection) to broadcast encrypted versions of all shares, and the degree- $2t$ shares of honest parties that received valid shares from R can still be reconstructed using these encryptions. Consequently, the only degree- $2t$ shares that may remain unavailable are those corresponding to parties unhappy with R – but since there can be at most only t such parties, this does not suffice to abort reconstruction. This completes a high-level overview of our construction.

In our construction, we use multi-string NIZKs (Appendix A.4) to enable parties to prove the correctness of their actions. In more detail, each party publishes its own CRS in the first round, and these strings are combined to form a joint CRS. As long as a majority of the individual CRS are generated honestly, proofs produced with respect to the joint CRS satisfy the standard zero-knowledge guarantees. The corresponding NIZK relations are defined below.

Relations for the mNIZKs. We utilize two relations, described below: $\mathcal{R}_{\text{sh}}$, for a proof of consistent committed-and-encrypted sharing (with configurable threshold r); and $\mathcal{R}_{\text{comp}}$, for the proof of the consistent evaluation of the equation $(1 - b)x_0 + bx_1$ on a set of committed shares.

$$\mathcal{R}_{\text{sh}} = \left\{ \begin{array}{l} \phi = (r, \{c_k, ct_k, \text{pk}_k\}_{k \in [n]}) \\ w = (\{o_k, \rho_{\text{enc},k}\}_{k \in [n]}, \rho_{\text{sh}}) \end{array} \middle| \begin{array}{l} ([a]_1, \dots, [a]_n) \leftarrow \text{share}(a, n, r; \rho_{\text{sh}}) \\ \forall k \in [n] : ct_k \leftarrow \text{Enc}(\text{pk}_k, [a]_k; \rho_{\text{enc},k}) \\ \forall k \in [n] : [a]_k \leftarrow \text{Decom}(o_k, c_k) \end{array} \right\}$$

$$\mathcal{R}_{\text{comp}} = \left\{ \begin{array}{l} \phi = (\langle x_b \rangle_k, c_k^b, c_k^{x_0}, c_k^{x_1}, c_k^0) \\ w = (([b]_k, o_k^b), ([x_0]_k, o_k^{x_0}), \\ \quad ([x_1]_k, o_k^{x_1}), (\langle 0 \rangle_k, o_k^0)) \end{array} \middle| \begin{array}{l} [b]_k \leftarrow \text{Decom}(o_k^b, c_k^b) \\ [x_0]_k \leftarrow \text{Decom}(o_k^{x_0}, c_k^{x_0}) \\ [x_1]_k \leftarrow \text{Decom}(o_k^{x_1}, c_k^{x_1}) \\ \langle 0 \rangle_k \leftarrow \text{Decom}(o_k^0, c_k^0) \\ \langle x_b \rangle_k = (1 - [b]_k) \cdot [x_0]_k + \\ [b]_k \cdot [x_1]_k + \langle 0 \rangle_k \end{array} \right.$$

Protocol Description. The formal description of our construction $\text{OPTI} = ((\text{OPTI}_1^R, \text{OPTI}_2^R), (\text{OPTI}_1^S, \text{OPTI}_2^S), ((\text{OPTI}_1^k, \text{OPTI}_2^k)_{k \in [n]}), \text{OPTI}_{\text{output}})$ appears in Figure 4.

OPTI Protocol

Building Blocks:

- Shamir secret sharing (**share**, **reconstruct**), as described in Appendix A.3
- A secure commitment scheme (**Com**, **Decom**), as described in Appendix A.2
- A secure public key encryption scheme (**KeyGen**, **Enc**, **Dec**, **KeyVer**), as described in Appendix A.1
- A multi-string NIZK scheme (**mNIZK.Gen**, **Prove**, **Verify**), as described in Appendix A.4

OPTI_1^R :

1. Compute $([b]_1, \dots, [b]_n) \leftarrow \text{share}(b, n, t+1; \rho^b)$.
2. For $k \in [n]$, compute $(o_k^b, c_k^b) \leftarrow \text{Com}([b]_k)$.
3. Send $([b]_k, o_k^b)$ to $\mathcal{P}_k$ and broadcast $\{c_k^b\}_{k \in [n]}$.

OPTI₁^S:

1. Compute $([x_0]_1, \dots, [x_0]_n) \leftarrow \text{share}(x_0, n, t+1; \rho^{x_0})$.
 Compute $([x_1]_1, \dots, [x_1]_n) \leftarrow \text{share}(x_1, n, t+1; \rho^{x_1})$.
 Compute $(\langle 0 \rangle_1, \dots, \langle 0 \rangle_n) \leftarrow \text{share}(0, n, 2t+1; \rho^0)$.
2. For $k \in [n]$:
 - Compute $(o_k^{x_0}, c_k^{x_0}) \leftarrow \text{Com}([x_0]_k)$.
 - Compute $(o_k^{x_1}, c_k^{x_1}) \leftarrow \text{Com}([x_1]_k)$.
 - Compute $(o_k^0, c_k^0) \leftarrow \text{Com}(\langle 0 \rangle_k)$.
 - Send $(([x_0]_k, o_k^{x_0}), ([x_1]_k, o_k^{x_1}), (\langle 0 \rangle_k, o_k^0))$ to $\mathcal{P}_k$.
3. Broadcast $\{(c_k^{x_0}, c_k^{x_1}, c_k^0)\}_{k \in [n]}$.

OPTI₁^k:

1. Compute $(\text{sk}_k, \text{pk}_k) \leftarrow \text{KeyGen}(\rho_k^{\text{gen}})$.
2. Compute $\text{crs}_k \leftarrow \text{mNIZK.Gen}(\rho_k^{\text{crs}})$.
3. Broadcast $(\text{pk}_k, \text{crs}_k)$.

OPTI₂^R:

1. Set $\overline{\text{crs}} = (\text{crs}_1, \dots, \text{crs}_n)$.
2. Set $\phi_{\text{sh}}^b = \{c_k^b\}_k, w_{\text{sh}}^b = (\{[b]_k\}_k, \{o_k^b\}_k, \rho^b)$.
3. Broadcast $\pi_{\text{sh}}^b \leftarrow \text{Prove}(\overline{\text{crs}}, \phi_{\text{sh}}^b, w_{\text{sh}}^b)$.

OPTI₂^S:

1. For $k \in [n]$:
 - Compute $ct_k^{x_0} \leftarrow \text{Enc}(\text{pk}_k, [x_0]_k; \rho_{\text{enc},k}^{x_0})$.
 - Compute $ct_k^{x_1} \leftarrow \text{Enc}(\text{pk}_k, [x_1]_k; \rho_{\text{enc},k}^{x_1})$.
 - Compute $ct_k^0 \leftarrow \text{Enc}(\text{pk}_k, \langle 0 \rangle_k; \rho_{\text{enc},k}^0)$.
2. Set $\overline{\text{crs}} \leftarrow (\text{crs}_1, \dots, \text{crs}_n)$.
3. Set $\phi_{\text{sh}}^{x_0} = (t+1, \{(c_k^{x_0}, ct_k^{x_0}, \text{pk}_k)\}_{k \in [n]}, w_{\text{sh}}^{x_0} = (\{(o_k^{x_0}, \rho_{\text{enc},k}^{x_0})\}_k, \rho^{x_0})$
 and broadcast $\pi_{\text{sh}}^{x_0} \leftarrow \text{Prove}(\overline{\text{crs}}, \phi_{\text{sh}}^{x_0}, w_{\text{sh}}^{x_0})$.
4. Set $\phi_{\text{sh}}^{x_1} = (t+1, \{(c_k^{x_1}, ct_k^{x_1}, \text{pk}_k)\}_{k \in [n]}, w_{\text{sh}}^{x_1} = (\{(o_k^{x_1}, \rho_{\text{enc},k}^{x_1})\}_k, \rho^{x_1})$
 and broadcast $\pi_{\text{sh}}^{x_1} \leftarrow \text{Prove}(\overline{\text{crs}}, \phi_{\text{sh}}^{x_1}, w_{\text{sh}}^{x_1})$.
5. Set $\phi_{\text{sh}}^0 = (2t+1, \{(c_k^0, ct_k^0, \text{pk}_k)\}_{k \in [n]}, w_{\text{sh}}^0 = (\{(o_k^0, \rho_{\text{enc},k}^0)\}_k, \rho^0)$
 and broadcast $\pi_{\text{sh}}^0 \leftarrow \text{Prove}(\overline{\text{crs}}, \phi_{\text{sh}}^0, w_{\text{sh}}^0)$.

OPTI₂^k:

1. Set $\overline{\text{crs}} \leftarrow (\text{crs}_1, \dots, \text{crs}_n)$.
2. Party k who received valid shares and openings from both S and R:
 - (a) Broadcast $\langle x_b \rangle_k = (1 - [b]_k) \cdot [x_0]_k + [b]_k \cdot [x_1]_k + \langle 0 \rangle_k$.
 - (b) Set $\phi_{\text{comp}}^{\langle x_b \rangle_k} = (\langle x_b \rangle_k, c_k^b, c_k^{x_0}, c_k^{x_1}, c_k^0)$,
 $w_{\text{comp}}^{\langle x_b \rangle_k} = (([b]_k, o_k^b), ([x_0]_k, o_k^{x_0}), ([x_1]_k, o_k^{x_1}), (\langle 0 \rangle_k, o_k^0))$.
 - (c) Broadcast $\pi_{\text{comp}}^{\langle x_b \rangle_k} \leftarrow \text{Prove}(\overline{\text{crs}}, \phi_{\text{comp}}^{\langle x_b \rangle_k}, w_{\text{comp}}^{\langle x_b \rangle_k})$.
3. Party k who received valid shares and openings from R only:
 - (a) Broadcast $(([b]_k, o_k^b), \text{sk}_k, \text{abort}_S)$.
4. Party k who received valid shares and openings from S only:
 - (a) Broadcast abort_R .
5. Party k who did not receive valid shares and openings from S or R:
 - (a) Broadcast $(\text{abort}_S, \text{abort}_R)$.

OPTI_{output}:

1. If over t parties sent abort_S or abort_R , output the corresponding abort.
2. Check $\text{Verify}(\overline{\text{crs}}, \phi_{\text{sh}}^b, \pi_{\text{sh}}^b) = 1$ and output abort_R otherwise.
3. Check the following:
 - $\text{Verify}(\overline{\text{crs}}, \phi_{\text{sh}}^{x_0}, \pi_{\text{sh}}^{x_0}) = 1$
 - $\text{Verify}(\overline{\text{crs}}, \phi_{\text{sh}}^{x_1}, \pi_{\text{sh}}^{x_1}) = 1$
 - $\text{Verify}(\overline{\text{crs}}, \phi_{\text{sh}}^0, \pi_{\text{sh}}^0) = 1$
 Output abort_S if any of the checks fail.
4. Let $\mathcal{S}_S, \mathcal{S}_R, \mathcal{S}_{S,R}, \mathcal{S}_\emptyset$ denote sets of indices of parties who aborted blaming receiver, who aborted blaming the sender S, who provided output and who aborted blaming both, respectively.
5. For $k \in \mathcal{S}_{S,R}$: check $\text{Verify}(\overline{\text{crs}}, \phi_{\text{comp}}^{\langle x_b \rangle_k}, \pi_{\text{comp}}^{\langle x_b \rangle_k}) = 1$ and remove k from $\mathcal{S}_{S,R}$ otherwise.
6. For $l \in \mathcal{S}_R$:
 - Check $\text{Decom}(o_l^b, c_l^b) = [b]_l$ and $\text{KeyVer}(\text{pk}_l, \text{sk}_l) = 1$, and remove l from $\mathcal{S}_R$ otherwise (moving on to the next l).
 - Compute $[x_0]_l \leftarrow \text{Dec}(\text{sk}_l, ct_l^{x_0})$.
 - Compute $[x_1]_l \leftarrow \text{Dec}(\text{sk}_l, ct_l^{x_1})$.
 - Compute $\langle 0 \rangle_l \leftarrow \text{Dec}(\text{sk}_l, ct_l^0)$.
 - Compute $\langle x_b \rangle_l = (1 - [b]_l) \cdot [x_0]_l + [b]_l \cdot [x_1]_l + \langle 0 \rangle_l$.
7. Output $x_b \leftarrow \text{reconstruct}(\{\langle x_b \rangle_k\}_{k \in \mathcal{S}_{S,R}}, \{\langle x_b \rangle_l\}_{l \in \mathcal{S}_R})$.

Fig. 4: OPTI Protocol

Theorem 2. *The OPTI construction in Figure 4 is secure as long as (share, reconstruct) is a secure secret sharing scheme (Appendix A.3), (Com, Decom) is a secure commitment scheme (Appendix A.2), (KeyGen, Enc, Dec, KeyVer) is a secure public key encryption scheme (Appendix A.1), and (mNIZK.Gen, Prove, Verify), is a secure multi-string NIZK scheme (Appendix A.4).*

3.3 Proof of Security

We prove Theorem 2 by proving each of its properties separately.

Claim. The OPTI construction in Figure 4 is sender-private (Definition 2).

Proof. Let $\mathcal{H}$ and $\mathcal{I}$ denote the sets of honest and corrupt parties, respectively. We assume a rushing adversary, meaning that in each round the adversary sends its messages only after receiving the messages of the honest parties, as in the security game.

In the first round, $\mathcal{A}$ obtains the sender's input shares held by the corrupt parties, $\{([x_0]_k, [x_1]_k, \langle 0 \rangle_k)\}_{k \in \mathcal{I}}$ together with the corresponding openings $\{(o_k^{x_0}, o_k^{x_1}, o_k^0)\}_{k \in \mathcal{I}}$ and the commitments to all parties' shares, $\{(c_k^{x_0}, c_k^{x_1}, c_k^0)\}_{k \in [n]}$. Learning honest parties' shares would require breaking the hiding property of the commitment scheme. Since $\mathcal{A}$ thus learns only the t corrupt parties' shares, it follows from security of the secret sharing scheme that this reveals no information about the underlying secret and therefore which two out of the four inputs provided earlier were used.

In the second round, $\mathcal{A}$ learns proofs of correctness for all committed and encrypted sharings, $\{\pi_{\text{sh}}^{x_0}, \pi_{\text{sh}}^{x_1}, \pi_{\text{sh}}^0\}$ and encryptions of shares, $\{(ct_k^{x_0}, ct_k^{x_1}, ct_k^0)\}_{k \in [n]}$. These proofs reveal no information due to the zero-knowledge property of the underlying mNIZK scheme. The encryptions to honest parties reveal no information due to the CPA-security of the encryption scheme. (Note that since the sender is honest in this game, no honest party will reveal its decryption key.)

The adversary further learns the honest parties' output shares $\{\langle x_b \rangle_k\}_{k \in \mathcal{H}}$ and the corresponding mNIZK proofs $\{\pi_{\text{comp}}^{\langle x_b \rangle_k}\}_{k \in \mathcal{H}}$. The former does not provide the adversary any advantage to extract information beyond x_b (which corresponds to $x_b^{c_b}$ in the sender privacy game), despite the fact that shares of b may be adversarially chosen, since computation of $\langle x_b \rangle_k$ involves masking with a threshold- $(2t + 1)$ share of 0. More specifically, the adversary does not learn any information about the honest party's shares corresponding to $x_{1-b}^{c_{1-b}}$. Next, due to the zero-knowledge property, $\mathcal{A}$ does not learn anything from the mNIZK proofs, beyond the correctness of the computation.

We also point out that the adversary cannot influence the protocol through the messages sent by the corrupt parties to learn additional information. In the first round, intermediary parties broadcast public keys and **crs** strings. The public keys are used only to encrypt shares of values already known to the adversary, and thus, malformed keys do not provide any advantage. Moreover,

the multi-string NIZK scheme preserves the zero-knowledge property even if up to t **crs** strings are malformed or have trapdoors known to the adversary.

This concludes the proof.

Claim. OPTI construction in figure 4 is private (Definition 3).

Proof. The arguments above already establish the privacy of the sender's input. In the privacy game, suppose that the adversary $\mathcal{A}$ can guess the order in which the inputs it provided were used as the sender's input in the protocol with probability at least $\frac{1}{2} + \lambda$, for some non-negligible λ . Then, by the previous arguments, one can construct an algorithm that breaks either the security of the secret-sharing scheme, the hiding property of the commitment scheme, the zero-knowledge property of the **mNIZK** scheme, or the CPA security of the encryption scheme. With similar arguments, one can also show that the receiver's communication to the adversary also does not reveal any information about b . This follows from security of secret sharing, since the adversary never learns more than t shares of b . In particular, this holds because of the hiding property of the commitment scheme and the fact that when the sender is honest, an honest $\mathcal{P}_k$ would never reveal its share $[b]_k$. This completes the proof.

Claim. OPTI construction in figure 4 achieves identifiability (Definition 4).

Proof. Let $\mathcal{H}$ and $\mathcal{I}$ denote the sets of honest and corrupt parties, respectively. First, we show that the output of an honest party $\mathcal{P}_k$ is never **abort** _{i} (except with negligible probability), where $i \in \mathcal{H}$ could denote **S** or **R**. We note that an honest $\mathcal{P}_k$ outputs **abort** _{i} only if either (1) more than t parties broadcast **abort** _{i} , or (2) a multi-string NIZK proof generated by $\mathcal{P}_i$ fails to verify. Case (1) cannot occur for an honest $\mathcal{P}_i$. This is because all honest parties verify their share openings against the public commitments, which would succeed for commitments generated by an honest $\mathcal{P}_i$ (due to correctness of the commitment scheme). Thus, there can be at most t complaints which could arise from corrupt parties. Case (2) cannot occur as well, due to the completeness of the multi-string NIZK, which holds even when up to t **crs** strings are chosen adversarially. Next, we analyze each of the cases of the identifiability game in Figure 3.

Case (a). When **S** and **R** are honest, given the arguments above, it remains to show that honest parties never output $x \neq x_b$. In this setting, $|\mathcal{S}_{\mathbf{S}, \mathbf{R}}| \geq 2t + 1$ holds (since all honest parties belong to $\mathcal{S}_{\mathbf{S}, \mathbf{R}}$) and their $2t$ -degree shares suffice to reconstruct x_b . The completeness of the multi-string NIZK proofs generated by honest parties in $\mathcal{S}_{\mathbf{S}, \mathbf{R}}$, together with the soundness of the proofs generated by potentially corrupt parties in $\mathcal{S}_{\mathbf{S}, \mathbf{R}}$, ensures that the shares used by honest parties for reconstruction of x_b are correct. Finally, even if some corrupt parties claim membership in $\mathcal{S}_{\mathbf{R}}$ and broadcast a secret key $\mathbf{sk}_l$ along with a share $[b]_l$ for $l \in \mathcal{I}$, the binding property of the commitment scheme and the key-verifiability property of the underlying encryption scheme ensure that only correct shares are used for reconstruction. Corrupt parties in $\mathcal{S}_{\mathbf{S}, \mathbf{R}}$ or $\mathcal{S}_{\emptyset}$ are unable to abort the reconstruction of x_b by the protocol design. This completes the analysis of case (a), showing that x_b is always reconstructed correctly.

To avoid redundancy, we do not analyze cases (b) and (c) separately, as they are subsumed by cases (a) and (d).

Case (d). We argue that either a corrupt party is identified as the cheater or a non- $\perp$ output is obtained by all honest parties. First, we observe that the adversary cannot send incorrectly computed messages in the first round, since the correctness of these messages is proven via **mNIZK** in the second round, and the soundness of **mNIZK** guarantees that such proofs can be generated only for correctly computed messages. Consequently, the only remaining room for misbehavior is the omission of private messages, or equivalently, sending invalid openings privately (that do not verify against the broadcast commitments), which can be treated as omissions for simplicity.

We now address the omission of messages. Let us denote the sizes of the following sets by

$$|\mathcal{S}_{S,R}| = t_{S,R}, \quad |\mathcal{S}_R| = t_R, \quad |\mathcal{S}_S| = t_S, \quad |\mathcal{S}_\emptyset| = t_\emptyset.$$

These values satisfy the following conditions:

1. $t_{S,R} + t_R + t_S + t_\emptyset = n \geq 3t + 1$
2. $\forall t_i$ for $t_i \in \{t_R, t_S, t_\emptyset\} \implies t_i \leq t$ (otherwise the output is **abort_S** or **abort_R**)
3. $t_R + t_\emptyset \leq t$, $t_S + t_\emptyset \leq t$ (otherwise the output is **abort_S** or **abort_R**)

This gives us a lower bound on t_R :

$$\begin{aligned} t_{S,R} + t_R + t_S + t_\emptyset &\geq 3t + 1 \\ \implies t_R &\geq 3t + 1 - t_{S,R} - (t_S + t_\emptyset) \geq 2t + 1 - t_{S,R} \end{aligned}$$

Recall that the final value $\langle x_b \rangle$ is obtained by combining two multiplications of shares of degree t , which requires at least $2t + 1$ shares. We receive $t_{S,R}$ shares of $\langle x_b \rangle$ from parties in $\mathcal{S}_{S,R}$ directly. Additionally, we retrieve the $\langle x_b \rangle$ shares of parties in $\mathcal{S}_R$ as follows: By decrypting the ciphertexts broadcast by **S** in the second round, using the secret key broadcast by the parties in $\mathcal{S}_R$ allows us to obtain the shares $[x_0]$, $[x_1]$, $\langle 0 \rangle$ of parties in $\mathcal{S}_R$. Recall that parties in $\mathcal{S}_R$ also broadcast the share $[b]$ (with the corresponding opening). Now, $[x_0]$, $[x_1]$, $[b]$, and $\langle 0 \rangle$, for the parties in $\mathcal{S}_R$ can be locally combined into t_R additional shares of $\langle x_b \rangle$. As $t_{S,R} + t_R \geq 2t + 1$, this yields a sufficient number of final shares to reconstruct the output. Any cheating by intermediary parties in the second round is handled identically to the case a).

Finally, we argue that all honest parties output the same value when running $\text{OPT}_{\text{output}}$. Any attempt to cheat or send malformed private messages is detected via the publicly verifiable commitments and **mNIZK** proofs. In addition, the correctness of the broadcast channel guarantees that all broadcast messages are identical for every honest party. This concludes the proof.

4 Definition of MPC with Identifiable Abort

We follow the real-world/ideal-world simulation paradigm and we adopt the security model of Cohen, Garay and Zikas [CGZ20].

Real world. An n -party protocol $\Pi = (\mathcal{P}_1, \dots, \mathcal{P}_n)$ is an n -tuple of probabilistic polynomial-time (PPT) interactive Turing machines (ITMs), where each party $\mathcal{P}_i$ is initialized with input $x_i \in \{0, 1\}^*$ and random coins $\rho_i \in \{0, 1\}^*$. We let $\mathcal{A}$ denote a special PPT ITM that represents the adversary and that is initialized with input that contains the identities of the corrupt parties, their respective private inputs and an auxiliary input.

The protocol is executed in rounds (i.e., the protocol is synchronous). Each round consists of a send phase and a receive phase, where parties can respectively send and receive messages. The communication in each round happens via a broadcast channel. During the execution of the protocol, the corrupt parties receive arbitrary instructions from the adversary $\mathcal{A}$, while the honest parties faithfully follow the instructions of the protocol. We consider the adversary $\mathcal{A}$ to be rushing, i.e., during every round, the adversary can see the messages the honest parties sent before producing messages for corrupt parties.

At the end of the protocol execution, the honest parties produce their outputs, and the adversary returns an arbitrary function of the corrupt parties' view. The view of a party during the execution consists of its input, random coins and the messages it sees during the execution.

Definition 5 (Real-world execution). Let $\Pi = (\mathcal{P}_1, \dots, \mathcal{P}_n)$ be an n -party protocol and let $\mathcal{I} \subseteq [n]$, of size at most t , denote the set of indices of the parties corrupted by $\mathcal{A}$. The joint execution of Π under $(\mathcal{A}, \mathcal{I})$ in the real world, on input vector $\mathbf{x} = (x_1, \dots, x_n)$, auxiliary input $\mathbf{aux}$ and the unary representation of the security parameter 1^λ , denoted $REAL_{\Pi, \mathcal{I}, \mathcal{A}(\mathbf{aux})}(\mathbf{x}, 1^\lambda)$, is defined as the output vector of $\mathcal{P}_1, \dots, \mathcal{P}_n$ and $\mathcal{A}(\mathbf{aux})$ resulting from the protocol interaction.

Ideal world.

Definition 6 (Ideal-world Computation). Let $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ be an n -party function and let $\mathcal{I} \subseteq [n]$, of size at most t , be the set of indices of the corrupt parties. Then, the joint ideal execution of f under $(\mathcal{S}, \mathcal{I})$ on input vector $\mathbf{x} = (x_1, \dots, x_n)$, auxiliary input $\mathbf{aux}$ to $\mathcal{S}$ and the unary representation of the security parameter 1^λ , denoted $IDEAL_{f, \mathcal{I}, \mathcal{S}(\mathbf{aux})}^{\text{id-abort}}(\mathbf{x}, 1^\lambda)$, is defined as the output vector of $\mathcal{P}_1, \dots, \mathcal{P}_n$ and $\mathcal{S}$ resulting from the following ideal process.

1. Parties send inputs to trusted party: An honest party $\mathcal{P}_i$ sends its input x_i to the trusted party. The simulator $\mathcal{S}$ may send to the trusted party arbitrary inputs for the corrupt parties. Let x'_i be the value actually sent as the input of party $\mathcal{P}_i$.
2. Trusted party speaks to simulator: The trusted party computes $(y_1, \dots, y_n) = f(x'_1, \dots, x'_n)$. If there are no corrupt parties proceed to step 3. Otherwise, the trusted party sends $\{y_i\}_{i \in \mathcal{I}}$ to $\mathcal{S}$.
3. Simulator $\mathcal{S}$ responds to the trusted party: If it chooses to abort, the simulator $\mathcal{S}$ can select a corrupt party $i^* \in \mathcal{I}$ who will be blamed, and send (abort, i^*) to the trusted party. If it does, we take $\mathcal{K} = [n] \setminus \mathcal{I}$.

Trusted party answers parties:

- (a) If the trusted party got abort from the simulator $\mathcal{S}$,

- i. It sets the abort message $\text{abortmsg} = (\perp, i^*)$.
- ii. The trusted party sends y_j to every party $\mathcal{P}_j$, $j \in [n] \setminus \mathcal{K}$. The trusted party sends abortmsg to every party $\mathcal{P}_j$, $j \in \mathcal{K}$.
- (b) Otherwise, it sends y to every $\mathcal{P}_j$, $j \in [n]$.
- 4. Outputs: Honest parties always output the message received from the trusted party, while the corrupt parties output nothing. The simulator $\mathcal{S}$ outputs an arbitrary function of the initial inputs $\{x_i\}_{i \in \mathcal{I}}$, the messages received by the corrupt parties from the trusted party and its auxiliary input.

Remark 1. The ideal-world computation for unanimous abort (UA) security is identical to the above, except that in Step 3, an aborting $\mathcal{S}$ does not provide a corrupt party index i^* . Consequently, the abortmsg is simply $\perp$ in this case.

Security Definitions. Now that we have described the real-world and the ideal-world experiment, we are ready to state the formal definition.

Definition 7. Let $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ be an n -party function. A protocol Π t -securely computes the function f with *id-abort* security if for every PPT real-world adversary $\mathcal{A}$ with auxiliary input aux , there exists a PPT simulator $\mathcal{S}$ such that for every $\mathcal{I} \subseteq [n]$ of size at most t , for all $\mathbf{x} \in (\{0, 1\}^*)^n$, for all $\lambda \in \mathbb{N}$, it holds that

$$\text{REAL}_{\Pi, \mathcal{I}, \mathcal{A}(\text{aux})}(\mathbf{x}, 1^\lambda) \approx_c \text{IDEAL}_{f, \mathcal{I}, \mathcal{S}(\text{aux})}^{\text{id-abort}}(\mathbf{x}, 1^\lambda)$$

5 Construction of Two-Round MPC with IA for $t < \frac{n}{3}$

Our approach to constructing our two-round IA protocol is to lift any two-round UA protocol in the plain model Π_{UA} (such as [ACGJ18]) to one that supports cheater identification. Our first step is to observe that the correctness of broadcast messages can be enforced via zero-knowledge proofs. As our protocol must operate in the plain model, we can not make use of standard NIZKs directly. Instead, we rely on multi-string NIZKs where the CRS is jointly sampled as part of the protocol: in the first round, each party publishes a CRS, and in the second round, proofs are generated with respect to the resulting joint CRS. Although proofs for the broadcast messages of both Rounds 1 and 2 can only be sent in the second round, this is still sufficient for cheater identification.

The main challenge is in detecting misbehavior in private channels in the first round (the second round can use only broadcast without loss of generality as discussed earlier). Consider a scenario where a corrupt $\mathcal{P}_j$ drops its private message to $\mathcal{P}_i$ in the first round. Whereas a UA protocol can handle this by having $\mathcal{P}_i$ simply broadcast ‘abort’ in the second round (leading all parties to abort unanimously) the IA setting requires more care. This is because other honest parties cannot tell whether $\mathcal{P}_i$ or $\mathcal{P}_j$ is corrupt in this scenario. We therefore require a mechanism that either identifies the cheating party in such scenarios, or enables the protocol to proceed without resolving this conflict explicitly—for instance by enabling $\mathcal{P}_i$ to compute its Round 2 message despite the missing private message.

We construct such a mechanism as follows: Each party $\mathcal{P}_i$ prepares a garbled circuit $\mathbb{GC}_i$, which garbles its next-message function for Round 2 of the protocol Π_{UA} . In more detail, the next-message function circuit takes as input $\mathcal{P}_i$'s private input and randomness, the Round 1 broadcast messages, and the Round 1 private messages received by $\mathcal{P}_i$ (denoted as $\{\text{msg}_{j \rightarrow i}^1\}_{j \in [n]}$) and computes $\mathcal{P}_i$'s second round message of Π_{UA} accordingly. For all inputs except the private messages, $\mathcal{P}_i$ can directly provide the corresponding wire labels. However, this may not be possible for the wire labels associated with private-message bits (e.g. as in the scenario above).

It is at this stage that we employ OPTI (Section 3) for verifiable transfer of the wire labels corresponding to the private message. As an illustrative example, consider a situation in which the first round of the protocol requires $\mathcal{P}_j$ to send a one bit message $\text{msg}_{j \rightarrow i}$ to $\mathcal{P}_i$ over their shared private channel. Let K^0 and K^1 be the labels on the corresponding wire in $\mathbb{GC}_i$. In the event that $\mathcal{P}_j$ withholds $\text{msg}_{j \rightarrow i}$ from $\mathcal{P}_i$, we ensure that OPTI delivers this message instead. In particular $\mathcal{P}_i$ plays the sender and provides both labels (K^0, K^1) as its input, and $\mathcal{P}_j$ plays the receiver with $\text{msg}_{j \rightarrow i}$ as its input. OPTI guarantees that either the correct label $K^{\text{msg}_{j \rightarrow i}}$ is reconstructed publicly by all parties, or a cheating party is identified.

Therefore, if no cheater is identified, all first-round private-message bits are correctly incorporated into the garbled circuits. We additionally use multi-string NIZKs to prove correctness of garbling. As a result, every party can evaluate the garbled circuits of all parties and obtain the corresponding Round 2 messages of Π_{UA} , ensuring correct completion of the protocol when no cheater is identified. This completes a high-level description of our protocol.

Before presenting the formal description of our IA protocol (Figure 5), we introduce some notation and describe a property we need from our garbling scheme. Specifically, we use a *garbling scheme with independent key generation* that allows us to pick keys independently of the circuit C . Therefore, we are able to first generate keys (i.e. labels) and later input them along with the circuit C into the **garble** function. Such a scheme can be instantiated with Yao's garbling scheme [Yao86]. We give a formal definition in Definition 12 (Appendix A.5).

Notation. Let $C_i(x_i, \rho_i, \{\text{msg}_j^1\}_{j \in [n]}, \{\text{msg}_{j \rightarrow i}^1\}_{j \in [n]})$ denote the Boolean circuit that takes party $\mathcal{P}_i$'s input x_i , randomness ρ_i , first-round broadcast messages $\text{msg}_1^1, \dots, \text{msg}_n^1$, and first-round private messages $\{\text{msg}_{j \rightarrow i}^1\}_{j \in [n]}$ and outputs msg_i^2 . For simplicity, we assume that (x_i, ρ_i) consists of z bits, and each first-round (broadcast or private) message is ℓ bits long, so each circuit has $L = z + 2n \cdot \ell$ input bits. Note that C_i is public. Let g be the size of a garbled C_i (we assume for simplicity each C_i has the same size). We assume that C_i has the following order:

1. Input wires on positions $[z]$ correspond to $\mathcal{P}_i$'s private input x_i and randomness ρ_i .
2. For $j \in [n]$, input wires on positions $\{z + \ell \cdot (j - 1) + \alpha \mid \alpha \in [\ell]\}$ correspond to $\mathcal{P}_j$'s first-round broadcast message msg_j^1 . We denote this range as $\{z + 1, \dots, M\}$, where $M = z + n \cdot \ell$.

3. For $j \in [n]$, input wires on positions $\{z + \ell \cdot (n + j - 1) + \alpha \mid \alpha \in [\ell]\}$ correspond to $\mathcal{P}_j$'s first-round private message $\text{msg}_{j \rightarrow i}^1$ sent to $\mathcal{P}_i$. We denote this range as $\{M + 1, \dots, L\}$.

Relations for mNIZKs In our IA protocol, we use multi-string NIZK for the following relation.

$$\mathcal{R}_{\text{IA}} = \left\{ \begin{array}{l} \left(\text{GC}_i, \text{msg}_i^1, \{K_i^\alpha\}_{\alpha \in [z]}, \right. \\ \phi = \{c_{i \rightarrow j}^\alpha\}_{\alpha \in [\ell], j \in [n]}, \\ \{(c_i^{(\alpha,0)}, c_i^{(\alpha,1)})\}_{\alpha \in \{z+1, \dots, L\}} \\ w = (\rho_i^{\text{GC}}, x_i, \rho_i, \{o_{i \rightarrow j}^\alpha\}_{\alpha \in [\ell], j \in [n]}, \\ \{(o_i^{(\alpha,0)}, o_i^{(\alpha,1)})\}_{\alpha \in \{z+1, \dots, L\}} \end{array} \left. \begin{array}{l} K_i \leftarrow \text{KGen}(1^\lambda, L, \rho_i^{\text{GC}}) \\ (\text{GC}_i) \leftarrow \text{garble}(1^\lambda, C_i, K_i, \rho_i^{\text{GC}}) \\ \text{msg}_i^1, \{\text{msg}_{i \rightarrow j}^1\}_{j \in [n]} \leftarrow \\ \quad \text{frst-msg}_i(x_i, \rho_i) \\ \forall \alpha \in [z] : K_i^\alpha = K_i^{\alpha, (x_i \parallel \rho_i)[\alpha]} \\ \forall \alpha \in \{z+1, \dots, L\}, b \in \{0, 1\} : \\ K_i^{(\alpha, b)} \leftarrow \text{Decom}(c_i^{(\alpha, b)}, o_i^{(\alpha, b)}) \\ \forall \alpha \in [\ell], j \in [n] : \\ \text{msg}_{i \rightarrow j}^1[\alpha] \leftarrow \text{Decom}(c_{i \rightarrow j}^\alpha, o_{i \rightarrow j}^\alpha) \end{array} \right\}$$

Remark 2. In the relation above, by a slight abuse of notation, the commitments to values actually refer to commitments to a secret sharing of those values (as used within the OPTI protocol). This is legitimate, since a set of commitments to shares is itself a secure commitment. We abuse notation in this way to simplify our relation statement.

Remark 3. Observe that in the protocol, each party commits to its labels without permuting their order. However, these commitments are never opened and are only used within the mNIZK proof, thereby keeping the position of the label that is later revealed hidden from the parties.

IA Protocol

Input: Each party $\mathcal{P}_i$ has input x_i and randomness ρ_i (which is split or expanded into randomness used for individual steps).

Building Blocks:

- Π_{OPTI} (Section 3.2)
- Π_{UA} is 2-round a honest-majority UA protocol securely computing f in the plain model (instantiated by [ACGJ18]), represented as a set of functions $\{\text{frst-msg}_i, \text{snd-msg}_i, \text{output}\}_{i \in [n]}$.

- A secure commitment scheme (Com, Decom), as described in Appendix A.2
- A multi-string NIZK scheme ($\text{mNIZK.Gen}, \text{Prove}, \text{Verify}$), as described in Appendix A.4
- A secure garbling scheme ($\text{KGen}, \text{garble}, \text{Eval}$) with independent key generation, as described in Appendix A.5

Round 1: Each $\mathcal{P}_i$ does the following:

- Compute $\text{crs}_i \leftarrow \text{mNIZK.Gen}(\rho_i^{\text{crs}})$
- Compute $\mathbf{K}_i \leftarrow \text{KGen}(1^\lambda, \mathbf{L}, \rho_i^{\text{gc}})$
- Let $\text{msg}_i^1, \{\text{msg}_{i \rightarrow j}^1\}_{j \in [n]} \leftarrow \text{frst-msg}_i(x_i, \rho_i)$ denote the first-round broadcast and private messages of $\mathcal{P}_i$ in Π_{UA} .
- Set labels corresponding to own input and randomness: $K_i^\alpha = K_i^{\alpha, (x_i \parallel \rho_i)[\alpha]}$ for $\alpha \in [z]$.
- Compute commitments of labels corresponding to the broadcast message indices: For $\alpha \in \{z+1, \dots, M\}$,
 - compute $(o_i^{(\alpha,0)}, c_i^{(\alpha,0)}) \leftarrow \text{Com}(K_i^{(\alpha,0)})$.
 - compute $(o_i^{(\alpha,1)}, c_i^{(\alpha,1)}) \leftarrow \text{Com}(K_i^{(\alpha,1)})$.
- Broadcast $(\text{crs}_i, \text{msg}_i^1, \{K_i^\alpha\}_{\alpha \in [z]}, \{(c_i^{(\alpha,0)}, c_i^{(\alpha,1)})\}_{\alpha \in \{z+1, \dots, M\}})$.
- For each bit of $\text{msg}_{j \rightarrow i}^1$ ($j \in [n]$), an OPTI instance is run with $\mathcal{P}_i$ as sender and $\mathcal{P}_j$ as receiver. Recall that the wire indices $\{M + (j-1) \cdot \ell + 1, \dots, M + j \cdot \ell\}$ correspond to $\text{msg}_{j \rightarrow i}^1$ in GC_i . For each $j \in [n]$ and each $\beta \in [\ell]$,
 - Let $\alpha := M + (j-1) \cdot \ell + \beta$.
 - Run round 1 of OPTI as S with inputs $(K_i^{(\alpha,0)}, K_i^{(\alpha,1)})$.
 - Let $((c_i^{(\alpha,0)}, c_i^{(\alpha,1)}), (o_i^{(\alpha,0)}, o_i^{(\alpha,1)}))$ denote the commitments and corresponding decommitments of the inputs $(K_i^{(\alpha,0)}, K_i^{(\alpha,1)})$, that are computed during round 1 of OPTI.
- For each bit of $\text{msg}_{i \rightarrow j}^1$ ($j \in [n]$), an OPTI instance is run with $\mathcal{P}_i$ as receiver and $\mathcal{P}_j$ as sender. Recall that the wire indices $\{M + (i-1) \cdot \ell + 1, \dots, M + i \cdot \ell\}$ correspond to $\text{msg}_{i \rightarrow j}^1$ in GC_j . For each $j \in [n]$ and $\beta \in [\ell]$:
 - Let $\alpha := M + (i-1) \cdot \ell + \beta$.
 - Run round 1 of OPTI as R with input $\text{msg}_{i \rightarrow j}^1[\beta]$.
 - Let $(c_{i \rightarrow j}^\beta, o_{i \rightarrow j}^\beta)$ denote the commitment and decommitment of the input $\text{msg}_{i \rightarrow j}^1[\beta]$ computed during round 1 of OPTI.
- Run round 1 of OPTI as an intermediary for all $n^2 \cdot \ell$ executions.

Round 2: Each $\mathcal{P}_i$ does the following:

- Set $\overline{\text{crs}} \leftarrow (\text{crs}_1, \dots, \text{crs}_n)$.
- Broadcast $\text{GC}_i \xleftarrow{\$} \text{garble}(C_i, \mathbf{K}_i, \rho_i^{\text{gc}})$, where C_i denotes the circuit corresponding to snd-msg_i .

- Set $\phi_{iA}^i = (\mathbf{GC}_i, \mathbf{msg}_i^1, \{K_i^\alpha\}_{\alpha \in [z]}, \{(c_i^{(\alpha,0)}, c_i^{(\alpha,1)})\}_{\alpha \in \{z+1, \dots, L\}}, \{c_{i \rightarrow j}^\alpha\}_{\alpha \in [\ell], j \in [n]})$ and $w_{iA}^i = (\rho_i^{\mathbf{GC}}, x_i, \rho_i, \{o_{i \rightarrow j}^\alpha\}_{\alpha \in [\ell], j \in [n]}, \{(o_i^{(\alpha,0)}, o_i^{(\alpha,1)})\}_{\alpha \in \{z+1, \dots, L\}})$.
- Broadcast $\pi_{iA}^i \leftarrow \text{Prove}(\overline{\mathbf{crs}}, \phi_{iA}^i, w_{iA}^i)$.
- Compute the labels for $\mathbf{GC}_i$ corresponding to first-round broadcast messages: For each $j \in [n], \beta \in [\ell]$, let $\alpha := z + \ell \cdot (j - 1) + \beta$, and set $K_i^\alpha = K_i^{(\alpha, \mathbf{msg}_j^1[\beta])}$.
- Broadcast $\{K_i^\alpha\}_{\alpha \in \{z+1, \dots, M\}}$.
- Run round 2 of the OPTI instances initiated in the first round i.e. for $j \in [n]$:
 - For $\beta \in [\ell]$, let $\alpha := M + (j - 1) \cdot \ell + \beta$, and run round 2 of OPTI as S with $R = \mathcal{P}_j$.
 - For $\beta \in [\ell]$, let $\alpha := M + (i - 1) \cdot \ell + \beta$, and run round 2 of OPTI as R with $S = \mathcal{P}_j$.
- Run round 2 of OPTI as an intermediary for all $n^2 \cdot \ell$ executions.

Output: Each party $\mathcal{P}_i$ does the following:

- Run $\text{OPTI}_{\text{output}}$ for all $n^2 \cdot \ell$ executions.
- If there was an instance of OPTI with an output $\mathbf{abort}_j$, output $\mathbf{abort}_j$.
- For $j \in [n]$: check $\text{Verify}(\overline{\mathbf{crs}}, \phi_{iA}^j, \pi_{iA}^j) = 1$ and output $\mathbf{abort}_j$ otherwise.
- For $j \in [n]$: compute $\mathbf{msg}_j^2 \leftarrow \text{Eval}(\mathbf{GC}_j, \{K_j^\alpha\}_{\alpha \in [L]})$ and output $\mathbf{abort}_j$ if evaluation fails.
- Compute $y \leftarrow \text{output}(\{\mathbf{msg}_j^1\}_{j \in [n]}, \{\mathbf{msg}_{j \rightarrow i}^1\}_{j \in [n]}, \{\mathbf{msg}_j^2\}_{j \in [n]})$

Fig. 5: IA Protocol

We are now ready to state our final theorem, the proof of which we summarize in Section 5.1, and give in full in Appendix B.

Theorem 3. *Let $(\text{Com}, \text{Decom})$ be a secure commitment scheme (Appendix A.2); $(\text{mNIZK.Gen}, \text{Prove}, \text{Verify})$ a secure multi-string NIZK scheme (Appendix A.4); $(\text{KGen}, \text{garble}, \text{Eval})$ a secure garbling scheme with independent key generation; and Π_{OPTI} be the 2-round OPTI protocol (Section 3.1) described in Figure 4. Let f be an efficiently computable function, and Π_{UA} be a 2-round protocol that securely computes f with unanimous abort in the presence of a malicious adversary that corrupts $t < n/3$ parties. Then, the 2-round IA construction in Figure 5 securely computes f with identifiable abort in the presence of a malicious adversary that corrupts $t < n/3$ parties.*

5.1 Proof of Security

Below we describe the simulator $\mathcal{S}$ for a real-world adversary $\mathcal{A}$.

Simulator $\mathcal{S}$

Let $\mathcal{H}$ and I denote the sets of honest and corrupt parties, respectively. Let $\mathcal{S}_{\text{UA}}$ denote the simulator for the underlying protocol Π_{UA} , and let f_{UA} and f_{IA} denote the ideal functionalities corresponding to Π_{UA} and Π_{IA} , respectively. Start by invoking $\mathcal{S}_{\text{UA}}$ and receive messages $\{(\text{msg}_i^1, \{\text{msg}_{i \rightarrow j}^1\}_{j \in I})\}_{i \in \mathcal{H}}$. Proceed with the following steps:

Round 1: On behalf of each honest party $\mathcal{P}_i \in \mathcal{H}$ do the following:

- Compute $(\text{crs}_i, \tau_i, \xi_i) \leftarrow \text{mNIZK.SE}_1(1^\lambda)$.
- Compute $\mathbf{K}_i \leftarrow \text{KGen}(1^\lambda, L, \rho_i^{\text{GC}})$ s.t. both labels in each pair are identical.
- Set each message in $\{\text{msg}_{i \rightarrow k}^1\}_{k \in \mathcal{H}}$ to 0.
- Complete the remainder of the first round as in the protocol.

Input Extraction: Upon completing the above steps, obtain first round broadcast messages $\{\text{msg}_j^1\}_{j \in I}$ from $\mathcal{A}$. To obtain the first-round private message for each execution of OPTI, where $S = \mathcal{P}_i$ for $i \in \mathcal{H}$ and $R = \mathcal{P}_j$ for $j \in I$ do the following steps:

- For $\beta \in [\ell]$, let $\{\text{msg}_{j \rightarrow i}^1[\beta]\}_{k \in \mathcal{H}}$ denote the shares the honest parties obtained from $R = \mathcal{P}_j$.
- For $\beta \in [\ell]$, reconstruct $\text{msg}_{j \rightarrow i}^1[\beta] \leftarrow \text{reconstruct}\{\text{msg}_{j \rightarrow i}^1[\beta]\}_{k \in \mathcal{H}}$. If reconstruction fails set $\text{msg}_{j \rightarrow i}^1 = \perp$.

For $j \in I, \beta \in [\ell]$, set $\text{msg}_{j \rightarrow i}^1 = \text{msg}_{j \rightarrow i}^1[1] \parallel \dots \parallel \text{msg}_{j \rightarrow i}^1[\ell]$ and send $\{(\text{msg}_j^1, \{\text{msg}_{j \rightarrow i}^1\}_{i \in \mathcal{H}})\}_{j \in I}$ to $\mathcal{S}_{\text{UA}}$. Upon receiving a request from $\mathcal{S}_{\text{UA}}$ to query f_{UA} with inputs $\{x_j\}_{j \in I}$, forward this message to f_{IA} and receive output y . Forward y to $\mathcal{S}_{\text{UA}}$ and receive $\{\text{msg}_i^2\}_{i \in \mathcal{H}}$ as a response.

Round 2: Set $\overline{\text{crs}} \leftarrow \{\text{crs}_i\}_{i \in \mathcal{H}}, \bar{\tau} \leftarrow \{\tau_i\}_{i \in \mathcal{H}}, \bar{\xi} \leftarrow \{\xi_i\}_{i \in \mathcal{H}}$ and for each $\mathcal{P}_i \in \mathcal{H}$ do the following:

- Broadcast $\text{GC}_i \xleftarrow{\$} \text{simGC}(C, \text{msg}_i^2, \mathbf{K}_i, \rho_i^{\text{GC}})$.
- Set $\phi_{\text{IA}}^i = (\text{GC}_i, \text{msg}_i^1, \{K_i^\alpha\}_{\alpha \in [z]}, \{(c_i^{(\alpha, 0)}, c_i^{(\alpha, 1)})\}_{\alpha \in \{z+1, \dots, L\}}, \{c_{i \rightarrow j}^\alpha\}_{\alpha \in [\ell], j \in [n]})$.
- Broadcast $\pi_{\text{IA}}^i \leftarrow \text{mNIZK.S}_2(\overline{\text{crs}}, \phi_{\text{IA}}^i, \bar{\tau})$.
- Complete the remainder of the second round as in the protocol.

Output:

- For each $j \in I$, if $\text{Verify}(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j) = \text{accept}$, then compute $w_{\text{IA}}^j \leftarrow \text{E}_2(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j, \bar{\xi})$. If extraction fails, send abort_j to the trusted party.
- For each $\mathcal{P}_i \in \mathcal{H}$ complete the output phase as in the protocol. If any execution results in abort_j , send abort_j to the trusted party.
- For every pair of parties, for every execution of OPTI do the following:
 - a) Let $\mathcal{P}_{i_1}$ be S with input b and $\mathcal{P}_{i_2}$ be R with input (x_0, x_1) , $\{i_1, i_2\} \in \mathcal{H}$. If there exists $\mathcal{P}_k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, \text{abort}_R, x\}$ for $x \neq x_b$, stop the simulation.

- b) Let $\mathcal{P}_i$ be S with input b and $\mathcal{P}_j$ be R with input (x_0, x_1) , $i \in \mathcal{H}, j \in I$. If there exists $\mathcal{P}_k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, x\}$ for $x \notin \{x_0, x_1\}$, stop the simulation.
- c) Let $\mathcal{P}_j$ be S with input b and $\mathcal{P}_i$ be R with input (x_0, x_1) , $i \in \mathcal{H}, j \in I$. If there exists $\mathcal{P}_k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_R\}$, stop the simulation.
- d) Let $\mathcal{P}_{j_1}$ be S with input b and $\mathcal{P}_{j_2}$ be R with input (x_0, x_1) , $\{j_1, j_2\} \in I$. If there exists $\mathcal{P}_k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k = \text{abort}$, stop the simulation.
- e) If there exist $\mathcal{P}_{i_1}, \mathcal{P}_{i_2} \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^{i_1} \neq \text{OPTI}_{\text{output}}^{i_2}$, stop the simulation.
- If no abort message was sent, send **continue** to the trusted party.
- Output the output of $\mathcal{A}$ and terminate.

Fig. 6: Simulator

We introduce a series of hybrid experiments to demonstrate that the joint distribution of the adversary’s output and the outputs of the honest parties in the ideal execution is computationally indistinguishable from the corresponding joint distribution obtained in a real protocol execution. In each experiment, the output is defined as the tuple consisting of the adversary’s output and the outputs of the honest parties.

We describe the sequence of hybrids in detail, and prove their indistinguishability, in Appendix B. Below, we summarize the hybrid steps.

Hybrid H_0 (Real World): Execute the protocol as in a real execution, using the honest parties’ inputs.

Hybrid H_1 : Same as Hybrid H_0 , except that for each honest party $\mathcal{P}_i$, the crs_i is generated as a part of a triple $(\text{crs}_i, \tau_i, \xi_i) \leftarrow \text{mNIZK.SE}_1(1^\lambda)$.

This hybrid is indistinguishable from the previous one by the reference string indistinguishability of the multi-string NIZK scheme.

Hybrid H_2 : Same as Hybrid H_1 , except that for each honest party $\mathcal{P}_i$, the proof for relation $\mathcal{R}_{\text{IA}}$ is generated using $\pi_{\text{IA}}^i \leftarrow \text{mNIZK.S}_2(\overline{\text{crs}}, \phi_{\text{IA}}^i, \bar{\tau})$.

This hybrid is indistinguishable from the previous one by the simulation indistinguishability of the multi-string NIZK scheme.

Hybrid H_3 : Same as Hybrid H_2 , except that for every corrupt party $\mathcal{P}_j \in I$, whenever $\text{Verify}(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j) = \text{accept}$, the simulator extracts a witness as $w_{\text{IA}}^j \leftarrow \text{E}_2(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j, \bar{\xi})$ and aborts if the extraction fails.

This hybrid is indistinguishable from the previous one by the simulation extractability of the multi-string NIZK scheme.

Hybrid H_4 : Same as Hybrid H_3 , except that for each honest party $\mathcal{P}_i$, the pair of labels $\mathbf{K}_i$ used in a given execution of OPTI are identical. We let this label be the “active” label for the message bit to which this execution of OPTI corresponds.

In honest parties’ garbled circuits, two different labels are still used (the circuit is garbled honestly, as before). Corrupt party messages are extracted from their secret sharings, and honest parties’ circuits are garbled so as to place the “active” label in the position corresponding to the messages used. *This hybrid is indistinguishable from the previous one by the sender privacy of OPTI.*

Hybrid H₅: Same as Hybrid H₄, except that for each honest party $\mathcal{P}_i$, the garbled circuit is generated using $\text{GC}_i \xleftarrow{\$} \text{simGC}(C, \text{msg}_i^2, \mathbf{K}_i, \rho_i^{\text{GC}})$.

This hybrid is indistinguishable from the previous one by the privacy of the garbled circuit scheme.

Hybrid H₆: Same as Hybrid H₅, except that honest parties’ inputs to OPTI corresponding to their private first-round messages to each other, i.e., $\{\text{msg}_{i \rightarrow k}^1\}_{i,k \in \mathcal{H}}$, are set to 0.

This hybrid is indistinguishable from the previous one by the privacy of OPTI.

Hybrid H₇: Same as Hybrid H₆, except that the honest parties’ messages to corrupt parties are replaced with simulated ones (generated using $\mathcal{S}_{\text{UA}}$).

This hybrid is indistinguishable from the previous one by the security of Π_{UA} .

Hybrid H₈ (Ideal World): Same as Hybrid H₇, except that for every pair of parties, for every execution of OPTI, $\mathcal{S}$ aborts if identifiability is violated.

This hybrid is indistinguishable from the previous one by the identifiability of OPTI.

In Hybrid H₈, the simulator no longer depends on any hidden information of the functionality, and honest parties output the correct output or agree on a corrupt party to blame, as per the definition of security with identifiable abort (Definition 7). Therefore, Hybrid H₈ is identically distributed as the ideal-world experiment, completing the proof.

Acknowledgements

Jure Sternad and Sophia Yakoubov are supported in part by the Danish Independent Research Council under Grant-ID DFF-2032-00122B and DFF-2064-00016B (YOSO). Divya Ravi is funded by NWO Talent Programme Veni (VI.Veni.242.295).

References

- ABT21. Benny Applebaum, Zvika Brakerski, and Rotem Tsabary. Perfect secure computation in two rounds. *SIAM J. Comput.*, 50(1):68–97, 2021.
- ACGJ18. Prabhanjan Ananth, Arka Rai Choudhuri, Aarushi Goel, and Abhishek Jain. Round-optimal secure multiparty computation with honest majority. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 395–424, August 2018.
- ACJ17. Prabhanjan Ananth, Arka Rai Choudhuri, and Abhishek Jain. A new approach to round-optimal secure multiparty computation. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 468–499, August 2017.

- AKP20. Benny Applebaum, Eliran Kachlon, and Arpita Patra. The round complexity of perfect MPC with active security and optimal resiliency. In *61st FOCS*, pages 1277–1284. IEEE Computer Society Press, November 2020.
- AKP23. Benny Applebaum, Eliran Kachlon, and Arpita Patra. The round complexity of statistical MPC with optimal resiliency. pages 1527–1536. ACM Press, 2023.
- AL10. Yonatan Aumann and Yehuda Lindell. Security against covert adversaries: Efficient protocols for realistic adversaries. *J. Cryptol.*, 23(2):281–343, 2010.
- BHP17. Zvika Brakerski, Shai Halevi, and Antigoni Polychroniadou. Four round secure computation without setup. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 645–677, November 2017.
- BL18. Fabrice Benhamouda and Huijia Lin. k-round multiparty computation from k-round oblivious transfer via garbled interactive circuits. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 500–532, April / May 2018.
- CCG⁺20. Arka Rai Choudhuri, Michele Ciampi, Vipul Goyal, Abhishek Jain, and Rafail Ostrovsky. Round optimal secure multiparty computation from minimal assumptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 291–319, November 2020.
- CDvdG88. David Chaum, Ivan Damgård, and Jeroen van de Graaf. Multiparty computations ensuring privacy of each party’s input and correctness of the result. In Carl Pomerance, editor, *CRYPTO’87*, volume 293 of *LNCS*, pages 87–119, August 1988.
- CGZ20. Ran Cohen, Juan A. Garay, and Vassilis Zikas. Broadcast-optimal two-round MPC. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 828–858, May 2020.
- DRSY23a. Ivan Damgård, Divya Ravi, Luisa Siniscalchi, and Sophia Yakoubov. Broadcast-optimal two round MPC with asynchronous peer-to-peer channels. In *LATINCRYPT*, volume 14168 of *Lecture Notes in Computer Science*, pages 87–106. Springer, 2023.
- DRSY23b. Ivan Damgård, Divya Ravi, Luisa Siniscalchi, and Sophia Yakoubov. Minimizing setup in broadcast-optimal two round MPC. In *EUROCRYPT 2023, Part II*, *LNCS*, pages 129–158, June 2023.
- ElG84. Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In G. R. Blakley and David Chaum, editors, *CRYPTO’84*, volume 196 of *LNCS*, pages 10–18, August 1984.
- GGHR14. Sanjam Garg, Craig Gentry, Shai Halevi, and Mariana Raykova. Two-round secure MPC from indistinguishability obfuscation. In Yehuda Lindell, editor, *TCC 2014*, volume 8349 of *LNCS*, pages 74–94, February 2014.
- GIKR02. Rosario Gennaro, Yuval Ishai, Eyal Kushilevitz, and Tal Rabin. On 2-round secure multiparty computation. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 178–193, August 2002.
- GJPR21. Aarushi Goel, Abhishek Jain, Manoj Prabhakaran, and Rajeev Raghunath. On communication models and best-achievable security in two-round MPC. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 97–128, November 2021.

- GLS15. S. Dov Gordon, Feng-Hao Liu, and Elaine Shi. Constant-round MPC with fairness and guarantee of output delivery. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 63–82, August 2015.
- GMPP16. Sanjam Garg, Pratyay Mukherjee, Omkant Pandey, and Antigoni Polychroniadou. The exact round complexity of secure computation. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 448–476, May 2016.
- GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
- GO07. Jens Groth and Rafail Ostrovsky. Cryptography in the multi-string model. In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 323–341, August 2007.
- GS18. Sanjam Garg and Akshayaram Srinivasan. Two-round multiparty secure computation from minimal assumptions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 468–499, April / May 2018.
- HLP11. Shai Halevi, Yehuda Lindell, and Benny Pinkas. Secure computation on the web: Computing without simultaneous interaction. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 132–150, August 2011.
- IKKP15. Yuval Ishai, Ranjit Kumaresan, Eyal Kushilevitz, and Anat Paskin-Cherniavsky. Secure computation with minimal interaction, revisited. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 359–378, August 2015.
- IKP10. Yuval Ishai, Eyal Kushilevitz, and Anat Paskin. Secure multiparty computation with minimal interaction. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 577–594, August 2010.
- KO04. Jonathan Katz and Rafail Ostrovsky. Round-optimal secure two-party computation. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 335–354, August 2004.
- KRY24. Sebastian Kolby, Divya Ravi, and Sophia Yakoubov. Constant-round YOSO MPC without setup. *IACR Commun. Cryptol.*, 1(3):30, 2024.
- MW16. Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key FHE. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 735–763, May 2016.
- Ped92. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *CRYPTO’91*, volume 576 of *LNCS*, pages 129–140, August 1992.
- PR18. Arpita Patra and Divya Ravi. On the exact round complexity of secure three-party computation. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 425–458, August 2018.
- Yao86. Andrew Chi-Chih Yao. How to generate and exchange secrets (extended abstract). In *27th FOCS*, pages 162–167. IEEE Computer Society Press, October 1986.

A Preliminaries

In this section, we describe the building blocks for our OPTI and MPC constructions.

A.1 Public Key Encryption

We take the definition of public key encryption directly from the work of Damgård *et al* [DRSY23b].

Definition 8 (Public-Key Encryption (PKE)). *A public-key encryption (PKE) scheme is a tuple of efficient algorithms $\text{PKE} = (\text{KGen}, \text{Enc}, \text{Dec})$ defined as follows.*

$\text{KGen}(1^\lambda) \rightarrow (\text{pk}, \text{sk})$: *The probabilistic algorithm KGen takes as input the security parameter $\lambda \in \mathbb{N}$, and outputs a public/secret key pair (pk, sk) .*

$\text{Enc}(\text{pk}, \text{msg}; \rho) \rightarrow \text{ct}$: *The probabilistic algorithm Enc takes as input the public key pk , a message msg , and implicit randomness ρ , and outputs a ciphertext $\text{ct} = \text{Enc}(\text{pk}, \text{msg}; \rho)$.*

$\text{Dec}(\text{sk}, \text{ct}) \rightarrow \text{msg}$: *The deterministic algorithm Dec takes as input the secret key sk and a ciphertext ct and outputs $\text{msg} = \text{Dec}(\text{sk}, \text{ct})$ which is either equal to some message msg or to an error symbol $\perp$.*

We require the following properties of a PKE scheme:

Correctness. We say that PKE satisfies correctness if for all $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$,

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, \text{msg})) = \text{msg}] = 1$$

(where the randomness is taken over the internal coin tosses of algorithm Enc).

CPA Security. We say that PKE satisfies semantic security if for all PPT adversaries $\mathcal{A}$, for $(\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}(1^\lambda)$, if $|\text{msg}_0| = |\text{msg}_1|$,

$$\Pr \left[\mathcal{A}(\text{pk}, \text{ct}) = b : \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda); b \leftarrow \{0, 1\}; \\ \text{ct} \leftarrow \text{Enc}(\text{pk}, \text{msg}_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

(where the randomness is taken over the internal coin tosses of $\mathcal{A}$, KGen and Enc).

Key Verifiability. We say that PKE satisfies key verifiability if there exists a deterministic polynomial-time algorithm KeyVer such that:

- For all $(\text{pk}, \text{sk}) \leftarrow \text{KGen}(1^\lambda)$,

$$\Pr[\text{KeyVer}(\text{pk}, \text{sk}) = 1] = 1.$$

- For all pk, sk such that (pk, sk) were not generated by KGen(1^λ),

$$\Pr[\text{KeyVer}(\text{pk}, \text{sk}) = 1] \leq \text{negl}(\lambda).$$

A.2 Commitment Scheme

We take the definition of a commitment scheme from the work of Damgård *et al* [DRSY23a].

A commitment scheme allows a party to commit to a value while keeping it hidden, and to later reveal the committed value with the guarantee that the commitment is binding [Ped92].

Definition 9 (Commitment Scheme). *A commitment scheme is a tuple of efficient algorithms $\text{COM} = (\text{Com}, \text{Decom})$ defined as follows.*

$\text{Com}(\text{msg}) \rightarrow (o, c)$: *The commitment algorithm Com takes as input a message $\text{msg} \in \{0, 1\}^\lambda$ and outputs a decommitment value $o \in \{0, 1\}^\lambda$ and a commitment value $c \in \{0, 1\}^\lambda$.*

$\text{Decom}(o, c) \rightarrow \{\text{msg}, \perp\}$: *The opening algorithm Decom takes as input a decommitment value $o \in \{0, 1\}^\lambda$ and a commitment value $c \in \{0, 1\}^\lambda$ and outputs either a message msg or $\perp$ in case o is not a valid decommitment for c .*

We require the following properties of a commitment scheme:

Correctness. We say that a commitment scheme COM satisfies correctness if for any message $\text{msg} \in \{0, 1\}^\lambda$ and $(o, c) \leftarrow \text{Com}(\text{msg})$, we have that

$$\Pr[\text{Decom}(c, o) = \text{msg}] \geq 1 - \text{negl}(\lambda).$$

Hiding. The hiding property requires that a commitment reveals nothing about the underlying message. More formally, we say that a commitment scheme COM satisfies hiding if for all PPT adversaries $\mathcal{A}$,

$$\Pr \left[b = b' \left| \begin{array}{l} (\text{msg}_0, \text{msg}_1) \leftarrow \mathcal{A}(1^\lambda); \\ b \leftarrow \{0, 1\}; \\ (o, c) \leftarrow \text{Com}(\text{msg}_b); \\ b' \leftarrow \mathcal{A}(c) \end{array} \right. \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Binding. The binding property requires that an adversary cannot open a commitment in two different ways. More formally, we say that a commitment scheme COM satisfies binding if for all PPT adversaries $\mathcal{A}$,

$$\Pr \left[\text{msg}_0, \text{msg}_1 \neq \perp, \text{msg}_0 \neq \text{msg}_1 \left| \begin{array}{l} (c, o_0, o_1) \leftarrow \mathcal{A}(1^\lambda); \\ \text{msg}_0 \leftarrow \text{Decom}(c, o_0); \\ \text{msg}_1 \leftarrow \text{Decom}(c, o_1) \end{array} \right. \right] \leq \text{negl}(\lambda).$$

Instantiation: ElGamal Commitment We use a commitment scheme based on ElGamal encryption [ElG84]. Let G be a cyclic group of prime order q generated by g , where $|q| = \lambda$. We assume that the group parameters (G, q, g) are publicly known.

The key generation algorithm samples a secret key $\text{sk} \leftarrow \mathbb{Z}_q$ and sets the public key $\text{pk} := g^{\text{sk}}$.

To commit to a message m , the committer samples randomness $r \leftarrow \mathbb{Z}_q$ and computes

$$\text{Com}(m; r) := (c_1, c_2) = (g^r, \text{pk}^r \cdot h(m)),$$

where $h : \mathbb{Z}_q \rightarrow G$ is an encoding of the message m into the group G . The opening of the commitment is (m, r) .

The scheme is perfectly binding and computationally hiding under the Decisional Diffie–Hellman (DDH) assumption in G .

Message Encoding. To encode messages as group elements, we use the following standard embedding into a prime-order subgroup of $\mathbb{Z}_p^*$. Let $p = 2q + 1$ be a safe prime and let $G \subseteq \mathbb{Z}_p^*$ denote the subgroup of quadratic residues of order q . Given $m \in \mathbb{Z}_q$, define $h : \mathbb{Z}_q \rightarrow G$ by

$$h(m) := \begin{cases} m + 1, & \text{if } (m + 1)^q \equiv 1 \pmod{p}, \\ -(m + 1) \bmod p, & \text{otherwise.} \end{cases}$$

Decoding $M \in G$ is performed by interpreting M as an integer in $\mathbb{Z}_p$ and outputting

$$m := \begin{cases} M - 1, & \text{if } M \leq q, \\ p - M - 1, & \text{otherwise.} \end{cases}$$

By Euler’s criterion, $(m + 1)^q \equiv 1 \pmod{p}$ holds if and only if $m + 1$ is a quadratic residue modulo p , which guarantees that $h(m) \in G$. The mapping h is efficiently computable and invertible.

A.3 Secret Sharing

Definition 10 (*t-out-of-n secret sharing scheme*). A *t-out-of-n secret sharing scheme* is a pair of algorithms $(\text{share}, \text{reconstruct})$, such that:

- $(s_1, \dots, s_n) \leftarrow \text{share}(s, n, t; \rho)$ is a randomized algorithm that, on input a secret s , outputs a set of n shares $(s_1, \dots, s_n)$ using randomness ρ and fixes the threshold t .
- $s \leftarrow \text{reconstruct}(s_{i_1}, \dots, s_{i_t})$ is a deterministic algorithm that on input t shares $(s_{i_1}, \dots, s_{i_t})$, where $(i_1, \dots, i_t) \subset [n]$, outputs the secret s .

A secret sharing scheme must satisfy the following properties:

Correctness: $\forall s, \forall T = (i_1, \dots, i_t) \subset [n], \forall \rho$ we have:

$$\Pr_{\text{share}(s, n, t; \rho) \rightarrow (s_1, \dots, s_n)} [\text{reconstruct}(s_{i_1}, \dots, s_{i_t}) = s] = 1.$$

Perfect Security: $\forall s, s', \forall I \subseteq \{1, \dots, n\}, \forall \rho$ where $|I| < t$, the following distributions are indistinguishable:

$$\begin{aligned} & \{(s_i : i \in I) \mid (s_1, \dots, s_n) \leftarrow \text{share}(s, n, t; \rho)\} \\ & \{(s'_i : i \in I) \mid (s'_1, \dots, s'_n) \leftarrow \text{share}(s', n, t; \rho)\} \end{aligned}$$

A.4 Multi-string Non-Interactive Zero-knowledge Proofs.

We follow the definition of a multi-string NIZK (mNIZK) from [KRY24] and restate it here for completeness. We refer to [KRY24] for further details.

Definition 11 (multi-string NIZK scheme). A mNIZK scheme for an NP relation $\mathcal{R}_{\mathcal{L}}$ has the following algorithms:

$\text{mNIZK.Gen}(1^\lambda) \rightarrow \mathbf{crs}$: An algorithm to generate a common reference string. In the multi-string model comprising of n common reference strings, we let $\overline{\mathbf{crs}} = (\mathbf{crs}_1, \dots, \mathbf{crs}_n)$ denote the vector of the n common reference strings.
 $\text{Prove}(\overline{\mathbf{crs}}, \phi, w) \rightarrow \pi$: An algorithm run by the prover that, given the vector of common reference strings $\overline{\mathbf{crs}}$, statement ϕ and the witness w outputs the proof π that $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$.
 $\text{Verify}(\overline{\mathbf{crs}}, \phi, \pi) \rightarrow \text{accept/reject}$: An algorithm that, given the vector of common reference strings $\overline{\mathbf{crs}}$, statement ϕ and the proof π verifies whether π proves the existence of a witness w such that $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$.

The rest of the algorithms are only necessary for proofs of security, and will not be used in the real world:

$S_1(1^\lambda) \rightarrow (\mathbf{crs}, \tau)$: A simulation algorithm that generates a simulated reference string and a simulation trapdoor.
 $S_2(\overline{\mathbf{crs}}, \phi, \bar{\tau}) \rightarrow \pi$: A simulation algorithm that, given the vector of common reference strings $\overline{\mathbf{crs}}$, statement ϕ and a vector $\bar{\tau}$ containing t_z (where t_z is a pre-defined threshold for the multi-string NIZK proof system) simulation trapdoors for common reference strings in $\overline{\mathbf{crs}}$, outputs a simulated proof of the existence of a witness w such that $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$.
 $E_1(1^\lambda) \rightarrow (\mathbf{crs}, \xi)$: A simulation algorithm that generates a simulated reference string and an extraction trapdoor.
 $E_2(\overline{\mathbf{crs}}, \phi, \pi, \bar{\xi}) \rightarrow w$: An extraction algorithm that, given the vector of common reference strings $\overline{\mathbf{crs}}$, statement ϕ , a valid proof π and a vector $\bar{\xi}$ containing t_s (where t_s is a pre-defined threshold for the multi-string NIZK proof system) extraction trapdoors for common reference strings in $\overline{\mathbf{crs}}$, outputs a witness w such that $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$.
 $SE_1(1^\lambda) \rightarrow (\mathbf{crs}, \tau, \xi)$: A simulation algorithm that outputs a simulated reference string, a simulation trapdoor and an extraction trapdoor such that $(\mathbf{crs}, \tau)$ is distributed as the output of S_1 , and $(\mathbf{crs}, \xi)$ is distributed as the output of E_1 .

Properties We require the following properties from a (t_c, t_s, t_z, n) multi-string NIZK proof system for an NP relation $\mathcal{R}_{\mathcal{L}}$ (as defined in the work [GO07]).

(t_c, t_s, t_z, n) -Completeness. Informally, this property requires that if at least t_c out of n common reference strings are honest, then the prover holding a witness for the statement should be able to create a convincing proof. More formally, for all non-uniform polynomial time adversaries $\mathcal{A}$,

$$\Pr \left[\text{Verify}(\overline{\text{crs}}, \phi, \pi) = 1 \mid \begin{array}{l} (\overline{\text{crs}}, \phi, w) \leftarrow \mathcal{A}^{\text{mNIZK.Gen}}(1^\lambda) \\ \pi \leftarrow \text{Prove}(\overline{\text{crs}}, \phi, w) \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

where mNIZK.Gen on query i output $\text{crs}_i \leftarrow \text{mNIZK.Gen}(1^\lambda)$, at least t_c of the crs_i 's generated by mNIZK.Gen are included and $\mathcal{A}$ outputs $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$, and negl is a negligible function in the security parameter λ .

We use multi-string NIZKs with perfect (t_c, t_s, t_z, n) -completeness for all $0 \leq t_c \leq n$ in our protocol. This means that even if the adversary chooses all common reference strings itself, we are guaranteed to output an acceptable proof when $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$.

(t_c, t_s, t_z, n) -Soundness. Informally, this property requires that if at least t_s out of n common random strings are honestly generated, then an adversary cannot forge the proof. The adversary gets to see possible choices of correctly generated common reference strings and can adaptively choose n of them. It may also include up to $n - t_s$ fake common reference strings it itself chooses. More formally, for all adversaries $\mathcal{A}$,

$$\Pr[\text{Verify}(\overline{\text{crs}}, \phi, \pi) = 1 \text{ and } \phi \notin \mathcal{L} : (\overline{\text{crs}}, \phi, \pi) \leftarrow \mathcal{A}^{\text{mNIZK.Gen}}(1^\lambda)] \leq \text{negl}(\lambda)$$

where mNIZK.Gen is an oracle that on query i outputs $\text{crs}_i \leftarrow \text{mNIZK.Gen}(1^\lambda)$, the adversary outputs $\overline{\text{crs}}$ such that at least t_s of the crs_i 's generated by mNIZK.Gen are included, and negl is a negligible function in the security parameter λ .

(t_c, t_s, t_z, n) -Zero Knowledge. Informally, this property requires that if t_z common reference strings are correctly generated, then the adversary learns nothing from the proof. As is standard in the zero-knowledge literature, we say that this is the case when the proof can be simulated given only the statement ϕ .

The definition of zero-knowledge is split into the following two parts.

Reference String Indistinguishability This property simply says that the adversary cannot distinguish real common reference strings from simulated reference strings. More formally, for all non-uniform polynomial time adver-

series $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(\mathbf{crs}) = 1 | \mathbf{crs} \leftarrow \text{mNIZK.Gen}(1^\lambda)] - \Pr[\mathcal{A}(\mathbf{crs}) = 1 | (\mathbf{crs}, \tau) \leftarrow \mathbf{S}_1(1^\lambda)] \right| \leq \text{negl}(\lambda)$$

for a negligible function negl in the security parameter λ .

(t_c, t_s, t_z, n)-Simulation Indistinguishability This property strengthens the standard definition of zero-knowledge and requires that even with access to the simulation trapdoors, the adversary cannot distinguish real proofs from simulated ones on a set of simulated reference strings. More formally, for all non-uniform polynomial time adversaries $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(\pi) = 1 | (\overline{\mathbf{crs}}, \bar{\tau}, \phi, w) \leftarrow \mathcal{A}^{\mathbf{S}_1}(1^\lambda); \pi \leftarrow \text{Prove}(\overline{\mathbf{crs}}, \phi, w)] - \Pr[\mathcal{A}(\pi) = 1 : (\overline{\mathbf{crs}}, \bar{\tau}, \phi, w) \leftarrow \mathcal{A}^{\mathbf{S}_1}(1^\lambda); \pi \leftarrow \mathbf{S}_2(\overline{\mathbf{crs}}, \bar{\tau}, \phi)] \right| \leq \text{negl}(\lambda),$$

where $\mathbf{S}_1$ on query i outputs $(\mathbf{crs}_i, \tau_i) \leftarrow \mathbf{S}_1(1^\lambda)$, the adversary outputs $(\phi, w) \in \mathcal{R}_{\mathcal{L}}$ and $\overline{\mathbf{crs}}, \bar{\tau}$ such that at least t_z of the $\mathbf{crs}_i$'s generated by $\mathbf{S}_1$ are included and $\bar{\tau}$ contains t_z simulation trapdoors τ_i corresponding to $\mathbf{crs}_i$'s that have been generated by the oracle $\mathbf{S}_1$, and negl is a negligible function in the security parameter λ .

(t_c, t_s, t_z, n)-Knowledge. Informally, this property requires the existence of probabilistic polynomial time algorithms $\mathbf{E}_1$ and $\mathbf{E}_2$ that can extract a witness from a valid proof.

Like the definition of zero-knowledge, the definition is split into two parts.

Reference String Indistinguishability For all non-uniform polynomial time adversaries $\mathcal{A}$,

$$\left| \Pr[\mathcal{A}(\mathbf{crs}) = 1 | \mathbf{crs} \leftarrow \text{mNIZK.Gen}(1^\lambda)] - \Pr[\mathcal{A}(\mathbf{crs}) = 1 | (\mathbf{crs}, \xi) \leftarrow \mathbf{E}_1(1^\lambda)] \right| \leq \text{negl}(\lambda)$$

where negl is a negligible function in the security parameter λ .

Extractability For all non-uniform polynomial time adversaries $\mathcal{A}$,

$$\Pr \left[\text{Verify}(\overline{\mathbf{crs}}, \phi, \pi) = 1, (\phi, w) \notin \mathcal{R}_{\mathcal{L}} \mid \begin{array}{l} (\overline{\mathbf{crs}}, \phi, \pi) \leftarrow \mathcal{A}^{\mathbf{E}_1}(1^\lambda) \\ w \leftarrow \mathbf{E}_2(\overline{\mathbf{crs}}, \phi, w, \bar{\xi}) \end{array} \right] \leq \text{negl}(\lambda)$$

where E_1 is an oracle that returns $(\mathbf{crs}_i, \xi_i) \leftarrow E_1(1^\lambda)$, $\bar{\xi}$ contains at least t_s ξ_i 's corresponding to the $\mathbf{crs}_i$'s generated by E_1 , and negl is a negligible function in the security parameter λ .

(t_c, t_s, t_z, n) -Simulation Soundness. Informally, this property requires that an adversary cannot prove any false statement even after seeing simulated proofs of arbitrary statements. More formally, for all non-uniform polynomial time adversaries $\mathcal{A}$, the following probability is negligible in the security parameter λ .

$$\Pr[\text{Verify}(\overline{\mathbf{crs}}, \phi, \pi) = 1, (\overline{\mathbf{crs}}, \phi, \pi) \notin Q, \phi \notin \mathcal{L} | (\overline{\mathbf{crs}}, \phi, \pi) \leftarrow \mathcal{A}^{S_1, S_2}(1^\lambda)]$$

where S_1 on query i returns $(\mathbf{crs}_i, \tau_i) \leftarrow S_1(1^\lambda)$, S_2 on query $(\overline{\mathbf{crs}}_j, \phi_j)$ returns $\pi_j \leftarrow S_2(\overline{\mathbf{crs}}_j, \bar{\tau}_j, \phi_j)$ with $\bar{\tau}_j$ having simulation trapdoors for the $\mathbf{crs}_i$'s generated by S_1 , the adversary produces $\overline{\mathbf{crs}}_j$ containing at least t_s $\mathbf{crs}_i$'s generated by S_1 , Q is the list of statements and corresponding proofs $(\overline{\mathbf{crs}}_j, \phi_j, \pi_j)$ in the queries to S_2 .

(t_c, t_s, t_z, n) -Simulation Extractability. Informally, this property requires that even after seeing many simulated proofs, whenever the adversary makes a new proof, we should be able to extract a witness. More formally, a multi-string NIZK proof system is (t_c, t_s, t_z, n) -simulation extractable if it has (t_c, t_s, t_z, n) -knowledge, is a (t_c, t_s, t_z, n) -NIZK proof (i.e. completeness, soundness and zero-knowledge hold for the relevant thresholds), and for all non-uniform polynomial time adversaries $\mathcal{A}$,

$$\Pr \left[\begin{array}{l} \text{Verify}(\overline{\mathbf{crs}}, \phi, \pi) = 1, \\ (\overline{\mathbf{crs}}, \phi, \pi) \notin Q, \\ (\phi, w) \notin \mathcal{R}_{\mathcal{L}} \end{array} \middle| \begin{array}{l} (\overline{\mathbf{crs}}, \phi, \pi) \leftarrow \mathcal{A}^{SE_1, S_2}(1^\lambda), \\ w \leftarrow E_2(\overline{\mathbf{crs}}, \phi, \pi, \bar{\xi}) \end{array} \right] \leq \text{negl}(\lambda)$$

where SE_1 on query i returns $(\mathbf{crs}_i, \xi_i)$ from $(\mathbf{crs}_i, \tau_i, \xi_i) \leftarrow SE_1(1^\lambda)$, S_2 on query $(\overline{\mathbf{crs}}_j, \phi_j)$ returns $\pi_j \leftarrow S_2(\overline{\mathbf{crs}}_j, \bar{\tau}_j, \phi_j)$ (where $\bar{\tau}_j$ contains t_z τ_i 's corresponding to $\mathbf{crs}_i$'s in $\overline{\mathbf{crs}}_j$ generated by SE_1), Q is the list of statements and corresponding proofs $(\overline{\mathbf{crs}}_j, \phi_j, \pi_j)$ made by S_2 , $\bar{\xi}$ contains the first t_s ξ_i 's generated by SE_1 corresponding to $\mathbf{crs}_i$'s in $\overline{\mathbf{crs}}$, and negl is a negligible function in the security parameter λ .

The above property of simulation extractability implies simulation-soundness.

Instantiation In our constructions, we use the $(0, t_s, t_z, n)$ multi-string NIZK with $t_s = t_z = \lceil (n+1)/2 \rceil$ of [GO07], which relies on enhanced trapdoor permutations and satisfies the properties outlined above.

A.5 Garbling Scheme

Definition 12. [DRSY23b] A garbling scheme with independent key generation $\mathcal{G}$ is a pair of algorithms $(\text{KGen}, \text{garble}, \text{Eval})$ defined as follows.

- $\mathbf{K} \leftarrow \text{KGen}(1^\lambda, L, \rho^{\text{GC}})$: The algorithm KGen takes as input a unary representation of the security parameter λ and a number of input wires and outputs L pairs of garbled labels $\mathbf{K} = (K_0^1, K_1^1, \dots, K_0^L, K_1^L)$. For simplicity, we assume that for every $i \in [L]$ and $b \in \{0, 1\}$, it holds that $K_b^i \in \{0, 1\}^\lambda$.
- $(\text{GC}) \xleftarrow{\$} \text{garble}(1^\lambda, C, \mathbf{K}, \rho^{\text{GC}})$: The algorithm garble takes as input a unary representation of the security parameter λ , a Boolean circuit $C : \{0, 1\}^L \rightarrow \{0, 1\}^m$ and keys $\mathbf{K}$, and outputs a garbled circuit GC .
- $y \leftarrow \text{Eval}(\text{GC}, K^1, \dots, K^L)$: The algorithm Eval takes as input the garbled circuit GC and L garbled labels $K^1, \dots, K^L$, and outputs a value $y \in \{0, 1\}^m$.

We require the following properties of a garbling scheme:

- **Perfect Correctness.** We say $\mathcal{G}$ satisfies *perfect correctness* if for any Boolean circuit $C : \{0, 1\}^L \rightarrow \{0, 1\}^m$ and $x = (x_1, \dots, x_L)$ it holds that

$$\Pr[\text{Eval}(\text{GC}, \mathbf{K}[x]) = C(x)] = 1,$$

where $\mathbf{K} \leftarrow \text{KGen}(1^\lambda, L, \rho^{\text{KGen}})$ with $\mathbf{K} = (K^{0,1}, K^{1,1}, \dots, K^{0,L}, K^{1,L})$, $\text{GC} \leftarrow \text{garble}(1^\lambda, C, \mathbf{K}, \rho^{\text{GC}})$, and $\mathbf{K}[x] := (K^{x_1,1}, \dots, K^{x_L,L})$.

- **Privacy.** We say that $\mathcal{G}$ satisfies *privacy* if there exists a simulator simGC such that for every PPT adversary $\mathcal{A}$, it holds that

$$\Pr[\text{GC}^{\text{priv}} = 1] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where GC^{priv} experiment is defined in the following figure:

The garbling scheme privacy experiment GC^{priv} .

1. $\mathcal{A}$ constructs a circuit $C : \{0, 1\}^L \rightarrow \{0, 1\}^m$, outputs an input $x = (x_1, \dots, x_L) \in \{0, 1\}^L$ and sends them to the challenger.
2. The challenger chooses a random bit $b \in \{0, 1\}$ and does the following:
 - (a) If $b = 0$:
 - Sample labels $\mathbf{K} \leftarrow \text{KGen}(1^\lambda, L, \rho^{\text{KGen}})$.
 - Sample garbled circuit $\text{GC} \leftarrow \text{garble}(1^\lambda, C, \mathbf{K}, \rho^{\text{GC}})$.
 - Set $K^i := K^{x_i,i}$ for $i \in [L]$.
 - (b) If $b = 1$:
 - Sample labels $\mathbf{K} \leftarrow \text{KGen}(1^\lambda, L, \rho^{\text{KGen}})$, s.t. only one label is sampled for every wire.
 - Sample simulated garbling $\text{GC} \leftarrow \text{simGC}(C, C(x), \mathbf{K}, \rho^{\text{GC}})$.
 - Set $K^i := K^{x_i,i}$ for $i \in [L]$.
3. The challenger sends $\text{GC}, K^1, \dots, K^L$ to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a bit b' .
5. $\mathcal{A}$ wins if $b = b'$, and we say that the experiment evaluates to 1 in this case.

B Hybrids for Proof of Theorem 3

Hybrid H_0 (Real World): Execute the protocol as in a real execution, using the honest parties' inputs.

Hybrid H_1 : Same as Hybrid H_0 , except that for each honest party $\mathcal{P}_i$, the crs_i is generated as a part of a triple $(\text{crs}_i, \tau_i, \xi_i) \leftarrow \text{mNIZK.SE}_1(1^\lambda)$.

Claim. Hybrids H_0 and H_1 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For $k \in \{0, \dots, h\}$, define hybrid experiment H_0^k as follows:

- For the first k honest parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_k}$, generate their crs as in H_1 : $(\text{crs}_i, \tau_i, \xi_i) \leftarrow \text{mNIZK.SE}_1(1^\lambda)$.
- For the remaining honest parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, generate their crs as in H_0 : $\text{crs}_i \leftarrow \text{mNIZK.Gen}(\rho_i^{\text{crs}})$.

Observe that H_0^0 is identical to H_0 , and H_0^h is identical to H_1 .

It therefore suffices to show that for every $k \in \{0, \dots, h-1\}$, hybrids H_0^k and H_0^{k+1} are computationally indistinguishable. For any fixed k , assume adversary $\mathcal{A}$ distinguishes these hybrids with non-negligible advantage. Then we construct an adversary $\mathcal{A}_{\text{mNIZK}}$ that internally runs $\mathcal{A}$ and simulates its view using the crs provided by the mNIZK challenger. If $\mathcal{A}$ distinguishes the hybrids, this implies that it can distinguish whether the crs_i for some honest party $\mathcal{P}_i$ was generated using mNIZK.Gen or as part of $(\text{crs}_i, \tau_i, \xi_i) \leftarrow \text{mNIZK.SE}_1(1^\lambda)$. $\mathcal{A}_{\text{mNIZK}}$ uses this distinguishing advantage to break reference string indistinguishability of the mNIZK scheme.

By applying this argument iteratively over all h pairs of hybrid experiments, we conclude that H_0 and H_1 are computationally indistinguishable.

Hybrid H_2 : Same as Hybrid H_1 , except that for each honest party $\mathcal{P}_i$, the proof for relation $\mathcal{R}_{\text{IA}}$ is generated using $\pi_{\text{IA}}^i \leftarrow \text{mNIZK.S}_2(\overline{\text{crs}}, \phi_{\text{IA}}^i, \tau)$.

Claim. Hybrids H_1 and H_2 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For $k \in \{0, \dots, h\}$, define hybrid experiment H_1^k as follows:

- For the first k honest parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_k}$, generate their proofs as in H_2 : $\pi_{\text{IA}}^i \leftarrow \text{mNIZK.S}_2(\overline{\text{crs}}, \phi_{\text{IA}}^i, \tau)$.
- For the remaining honest parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, generate their proofs as in H_1 : $\pi_{\text{IA}}^i \leftarrow \text{mNIZK.Prove}(\overline{\text{crs}}, \phi_{\text{IA}}^i, w^i)$.

Observe that H_1^0 is identical to H_1 , and H_1^h is identical to H_2 .

It therefore suffices to consider an arbitrary $k \in \{0, \dots, h-1\}$ and argue that H_1^k is indistinguishable from H_1^{k+1} . The adversary $\mathcal{A}$ already has access to the simulated reference strings and the corresponding simulation trapdoors for all corrupt parties (it can generate them using mNIZK.S_1). Suppose $\mathcal{A}$ detects this change with non-negligible advantage. We then build a distinguisher $\mathcal{A}_{\text{mNIZK}}$ for the simulation indistinguishability experiment of mNIZK

by running $\mathcal{A}$ and answering all queries as in the hybrid, except that the pivot honest party's proof is obtained from the challenger. If $\mathcal{A}$ distinguishes the hybrids, it follows that $\mathcal{A}_{\text{mNIZK}}$ distinguishes whether π_{IA}^i was produced by **Prove** or by S_2 , contradicting simulation indistinguishability. By applying this argument iteratively over all h pairs of hybrid experiments, we conclude that H_1 and H_2 are computationally indistinguishable.

Hybrid H_3 : Same as Hybrid H_2 , except that for every corrupt party $\mathcal{P}_j \in I$, whenever $\text{Verify}(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j) = \text{accept}$, the simulator extracts a witness as $w_{\text{IA}}^j \leftarrow \text{E}_2(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j, \bar{\xi})$ and aborts if the extraction fails.

Claim. Hybrids H_2 and H_3 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For $k \in \{0, \dots, h\}$, define hybrid experiment H_2^k as follows:

- For the first k honest parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_k}$, act as in Hybrid H_3 , i.e., whenever a corrupt party $\mathcal{P}_j \in I$ produces a proof π_{IA}^j such that $\text{Verify}(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j) = \text{accept}$, extract a witness $w_{\text{IA}}^j \leftarrow \text{E}_2(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j, \bar{\xi})$ and abort with message abort_j if extraction fails;
- For the remaining honest parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, act as in Hybrid H_2 , i.e., do not extract.

Observe that H_2^0 is identical to H_2 , and H_2^h is identical to H_3 . It therefore suffices to show that for every $k \in \{0, \dots, h-1\}$, hybrids H_2^k and H_2^{k+1} are computationally indistinguishable.

Assume, for contradiction, that $\mathcal{A}$ distinguishes Hybrids H_2 and H_3 with non-negligible advantage. We show how to turn $\mathcal{A}$ into an adversary $\mathcal{A}_{\text{mNIZK}}$ for the simulation extractability experiment. The reduction runs $\mathcal{A}$ while answering all simulated **crs** and simulated proof queries via SE_1 and S_2 , thereby producing an execution distributed as required by the hybrid. Now note that the only way the two hybrids differ is when $\mathcal{A}$ causes an accepting corrupt-party proof for which the extraction fails. Hence, if $\mathcal{A}$ distinguishes, it must (with non-negligible probability) produce some $(\phi_{\text{IA}}^j, \pi_{\text{IA}}^j)$ such that $\text{Verify}(\overline{\text{crs}}, \phi_{\text{IA}}^j, \pi_{\text{IA}}^j) = \text{accept}$ but extraction fails. In that case $\mathcal{A}_{\text{mNIZK}}$ outputs this triple, violating simulation extractability.

In particular, this also implies that the adversary cannot violate simulation soundness.

Hybrid H_4 : Same as Hybrid H_3 , except that for each honest party $\mathcal{P}_i$, the pair of labels in $\mathbf{K}_i$ used in a given execution of **OPTI** are identical. We let this label be the “active” label for the message bit to which this execution of **OPTI** corresponds.

In honest parties' garbled circuits, two different labels are still used (the circuit is garbled honestly, as before). Corrupt party messages are extracted from their secret sharings, and honest parties' circuits are garbled so as to place the “active” label in the position corresponding to the messages used.

Claim. Hybrids H_3 and H_4 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For simplicity, assume that the private messages are of size 1 bit. For $k \in \{0, \dots, h\}$, define hybrid experiment H_3^k as follows:

- For parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_{k-1}}$, generate labels as in Hybrid H_4 .
- For parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, generate labels as in Hybrid H_3 .
- For the party $\mathcal{P}_{i_k}$, generate labels using the sender-privacy challenger of OPTI as in the game: whenever a label pair for some wire is needed, sample K_0, K_1 and submit the two candidate sender inputs (K_0, K_1) and (K_0, K_0) to the challenger (note that depending on the bit chosen by the challenger, the labels are either picked as in H_3 or H_4). Use the messages returned by the challenger to complete the simulation for $\mathcal{A}$.

Observe that H_3^0 is identical to H_3 and H_3^h is identical to H_4 . It therefore suffices to show that for every $k \in \{0, \dots, h-1\}$, hybrids H_3^k and H_3^{k+1} are computationally indistinguishable.

Assume adversary $\mathcal{A}$ could distinguish between the two hybrids. Then we can construct an adversary $\mathcal{A}_{\text{OPTI}}$ that breaks the sender privacy property of OPTI. The adversary $\mathcal{A}_{\text{OPTI}}$ internally runs $\mathcal{A}$ and provides it with a simulated view of the hybrid execution. If the challenger uses input (K_0, K_1) , then the labels of $\mathcal{P}_{i_k}$ are distributed as in Hybrid H_3 ; if the challenger uses input (K_0, K_0) , then the labels of $\mathcal{P}_{i_k}$ are distributed as in Hybrid H_4 . Therefore, $\mathcal{A}_{\text{OPTI}}$ can use $\mathcal{A}$'s output to distinguish the challenger's choice with the same non-negligible advantage, contradicting sender privacy of OPTI.

By applying this argument iteratively over $k = 0, \dots, h-1$, we conclude that Hybrids H_3 and H_4 are computationally indistinguishable.

Hybrid H_5 : Same as Hybrid H_4 , except that for each honest party $\mathcal{P}_i$, the garbled circuit is generated using $\text{GC}_i \xleftarrow{\$} \text{simGC}(C, \text{msg}_i^2, \mathbf{K}_i, \rho_i^{\text{GC}})$, where for each $i \in \mathcal{H}$, msg_i^2 is computed honestly as $\text{msg}_i^2 \leftarrow \text{snd-msg}(x_i, \rho_i, \{\text{msg}_j^1\}_{j \in [n]}, \{\text{msg}_{j \rightarrow i}^1\}_{j \in [n]})$.

Claim. Hybrids H_4 and H_5 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For $k \in \{0, \dots, h\}$, define hybrid experiment H_4^k as follows:

- For parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_{k-1}}$, generate garbled circuits as in Hybrid H_5 :
 $\text{GC}_i \xleftarrow{\$} \text{simGC}(C, \text{msg}_i^2, \mathbf{K}_i, \rho_i^{\text{GC}})$.
- For parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, generate garbled circuits as in Hybrid H_4 :
 $\text{GC}_i \xleftarrow{\$} \text{garble}(C, \mathbf{K}_i, \rho_i^{\text{GC}})$.
- For the party $\mathcal{P}_{i_k}$, generate the garbled circuit using the privacy challenger of the garbling scheme as in the game GC^{priv} : submit the circuit C and the corresponding input used in the protocol execution, and obtain from the challenger either (i) an honestly generated garbled circuit and matching labels, or (ii) a simulated garbled circuit and matching labels (depending on the challenger's hidden bit).

Observe that H_4^0 is identical to H_4 and H_4^h is identical to H_5 . It therefore suffices to show that for every $k \in \{0, \dots, h-1\}$, hybrids H_4^k and H_4^{k+1} are computationally indistinguishable.

Assume $\mathcal{A}$ distinguishes Hybrids H_4^k and H_4^{k+1} for some $k \in \{0, \dots, h\}$. We build an adversary $\mathcal{A}_{GC}$ for the garbling-privacy experiment. The reduction runs $\mathcal{A}$ and simulates the hybrid execution, generating all parties' garbled circuits honestly except for the pivot party $\mathcal{P}_{i_k}$. For this pivot, $\mathcal{A}_{GC}$ submits the circuit C and the relevant input to the privacy challenger and uses the returned $(GC_{i_k}, \mathbf{K}_{i_k})$ in the simulation.

If the challenger returns a garbled circuit, generated as $GC_i \xleftarrow{\$} \text{garble}(C, \mathbf{K}_{i_k}, \rho_{i_k}^{GC})$, then the garbled circuit of $\mathcal{P}_{i_k}$ is distributed as in Hybrid H_4^k ; if the challenger returns a simulated garbled circuit, generated as $GC_i \xleftarrow{\$} \text{simGC}(C, \text{msg}_{i_k}^2, \mathbf{K}_{i_k}, \rho_{i_k}^{GC})$, then it is distributed as in Hybrid H_4^{k+1} . Consequently, any distinguishing advantage of $\mathcal{A}$ transfers to $\mathcal{A}_{GC}$, contradicting privacy of the garbling scheme.

By applying this argument iteratively over $k = 0, \dots, h-1$, we conclude that H_4 and H_5 are computationally indistinguishable.

Finally, note that in Hybrid H_5 the simulated garbled circuits may no longer correspond to a true statement for the relation proven by the associated mNIZK proofs. However, this does not affect the above argument, since the mNIZK proofs were already replaced by simulated proofs in Hybrid H_2 , and hence the adversary cannot detect any inconsistency between the simulated garbled circuits and the simulated proofs.

Hybrid H_6 : Same as Hybrid H_5 , except that honest parties' inputs to OPTI corresponding to their private first-round messages to each other, i.e., $\{\text{msg}_{i \rightarrow k}^1\}_{i,k \in \mathcal{H}}$, are set to 0.

Claim. Hybrids H_5 and H_6 are computationally indistinguishable.

Proof. Let $\mathcal{H} = \{\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_h}\}$ denote the set of honest parties, where $h = |\mathcal{H}|$. For simplicity, assume that the private messages are of size 1 bit. For $k, l \in \{0, \dots, h\}$, define hybrid experiment $H_5^{(k,l)}$ as follows:

- For parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_{k-1}}$, generate the private messages they send to other honest parties as in Hybrid H_6 .
- For parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$, generate the private messages they send to other honest parties as in Hybrid H_5 .
- For the party $\mathcal{P}_{i_k}$, generate the private messages it sends to parties $\mathcal{P}_{i_1}, \dots, \mathcal{P}_{i_{k-1}}$ as in Hybrid H_6 and the private messages it sends to parties $\mathcal{P}_{i_{k+1}}, \dots, \mathcal{P}_{i_h}$ as in Hybrid H_5 . For the private message sent to party $\mathcal{P}_{i_l}$, obtain it from the challenger in the privacy game of OPTI as follows. Submit $(K_l^{(\alpha,0)}, K_l^{(\alpha,1)})$ to the challenger, where α denotes the wire position corresponding to the private message $\text{msg}_{i_k \rightarrow i_l}^1$ in the garbled circuit of $\mathcal{P}_{i_l}$. Use the messages returned by the challenger to complete the simulation for $\mathcal{A}$.

Observe that $H_5^{(0,0)}$ is identical to H_5 and $H_5^{(h,h)}$ is identical to H_6 . It therefore suffices to show that for every $k, l \in \{0, \dots, h-1\}$ the following hold:

- Hybrids $H_5^{(k,l)}$ and $H_5^{(k,l+1)}$ are computationally indistinguishable.
- Hybrids $H_5^{(k,h)}$ and $H_5^{(k+1,0)}$ are computationally indistinguishable.

Assume adversary $\mathcal{A}$ could distinguish between the two hybrids. Then we can construct an adversary $\mathcal{A}_{\text{OPTI}}$ that breaks the privacy property of OPTI. The adversary $\mathcal{A}_{\text{OPTI}}$ internally runs $\mathcal{A}$ and provides it with a simulated view of the hybrid execution. Depending on the challenger's bit, the messages are either distributed as in H_5 or as in H_6 . Therefore, $\mathcal{A}_{\text{OPTI}}$ can use $\mathcal{A}$'s output to distinguish the challenger's choice with the same non-negligible advantage, contradicting privacy of OPTI.

By applying this argument iteratively over all choices of k and l , we conclude that hybrids H_5 and H_6 are computationally indistinguishable.

Hybrid H_7 : Same as Hybrid H_6 , except that the honest parties' messages to corrupt parties are replaced as follows:

- Obtain the honest parties' first-round messages from $\mathcal{S}_{\text{UA}}$.
- Extract each corrupt party's private message from the secret sharings in OPTI.
- Invoke $\mathcal{S}_{\text{UA}}$ using the broadcast and private messages from corrupt to honest parties $\{(\text{msg}_j^1, \{\text{msg}_{j \rightarrow i}^1\}_{i \in \mathcal{H}})\}_{j \in \mathcal{I}}$.
- Obtain the extracted corrupt parties' inputs from the ideal functionality query issued by $\mathcal{S}_{\text{UA}}$.
- Query the ideal functionality to obtain output y , forward y to $\mathcal{S}_{\text{UA}}$, and receive the honest parties' second-round broadcast messages.

Claim. Hybrids H_6 and H_7 are computationally indistinguishable.

Proof. Assume towards contradiction that there exists a non-uniform *PPT* adversary $\mathcal{A}$ that distinguishes Hybrids H_6 and H_7 with non-negligible probability. We use $\mathcal{A}$ to construct an adversary $\mathcal{A}_{\text{UA}}$ that breaks the UA-security of the underlying protocol Π_{UA} .

Recall that in Hybrid H_7 , the honest parties' messages sent to corrupt parties are generated using the simulator $\mathcal{S}_{\text{UA}}$, whereas in Hybrid H_6 these messages are produced according to an honest execution of Π_{UA} .

The adversary $\mathcal{A}_{\text{UA}}$ internally runs $\mathcal{A}$ and simulates for it the execution of the protocol as follows. The adversary $\mathcal{A}_{\text{UA}}$ interacts with the real or simulated honest parties in an execution of Π_{UA} ; it uses the messages it receives from those real or simulated honest parties to run the code of Hybrid 7. If the honest parties are real, this will be distributed exactly as Hybrid 6; if they are simulated, this will be distributed exactly as Hybrid 7. Once $\mathcal{A}$ makes its guess, $\mathcal{A}_{\text{UA}}$ makes the same guess; it distinguishes correctly exactly when $\mathcal{A}$ does.

Therefore, any non-negligible distinguishing advantage of $\mathcal{A}$ between Hybrids H_6 and H_7 directly translates into a non-negligible advantage for $\mathcal{A}_{\text{UA}}$ in distinguishing between the real and ideal executions in the UA-security experiment for Π_{UA} . This contradicts the assumed IA-security of Π_{UA} .

Hence, Hybrids H_6 and H_7 are computationally indistinguishable.

Hybrid H_8 : Same as Hybrid H_7 , except that for every pair of parties, for every execution of OPTI, S does the following:

- a) For any execution in which $\mathcal{P}_{i_1}$ acts as S with input b and $\mathcal{P}_{i_2}$ acts as R with input (x_0, x_1) , where $\{i_1, i_2\} \subseteq \mathcal{H}$, if there exists $k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, \text{abort}_R, x\}$ for $x \neq x_b$, stop the simulation.
- b) For any execution in which $\mathcal{P}_i$ acts as S with input b and $\mathcal{P}_j$ acts as R with input (x_0, x_1) , where $i \in \mathcal{H}$ and $j \in I$, if there exists $k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_S, x\}$ for $x \notin \{x_0, x_1\}$, stop the simulation.
- c) For any execution in which $\mathcal{P}_j$ acts as S with input b and $\mathcal{P}_i$ acts as R with input (x_0, x_1) , where $i \in \mathcal{H}$ and $j \in I$, if there exists $k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k \in \{\text{abort}, \text{abort}_R\}$, stop the simulation.
- d) For any execution in which $\mathcal{P}_{j_1}$ acts as S with input b and $\mathcal{P}_{j_2}$ acts as R with input (x_0, x_1) , where $\{j_1, j_2\} \subseteq I$, if there exists $k \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^k = \text{abort}$, stop the simulation.
- e) If there exist $\mathcal{P}_{i_1}, \mathcal{P}_{i_2} \in \mathcal{H}$ such that $\text{OPTI}_{\text{output}}^{i_1} \neq \text{OPTI}_{\text{output}}^{i_2}$, stop the simulation.

Claim. Hybrids H_7 and H_8 are computationally indistinguishable.

Proof. Notice that cases (a)-(e) listed above correspond exactly to the events under which an adversary wins the OPTI identifiability game.

Assume towards contradiction that there exists an adversary $\mathcal{A}$ that distinguishes Hybrids H_7 and H_8 with non-negligible advantage. We construct an adversary $\mathcal{A}_{\text{OPTI}}$ that breaks the identifiability property of OPTI.

The adversary $\mathcal{A}_{\text{IA}}$ internally runs $\mathcal{A}$ and simulates the view of the protocol execution exactly as in Hybrid H_7 . Whenever the simulation reaches an execution of OPTI, $\mathcal{A}_{\text{IA}}$ forwards the corresponding messages to the identifiability challenger and uses the responses returned by the challenger to continue the simulation.

If during the simulation the transcript satisfies any of the conditions (a)-(e), then $\mathcal{A}_{\text{IA}}$ outputs the transcript as a successful winning instance for the identifiability game. Observe that Hybrid H_9 aborts exactly in this event, however, by the identifiability property of OPTI, the probability that an adversary produces a transcript satisfying any of the winning conditions is negligible. Hence, the probability that Hybrid H_8 aborts while Hybrid H_7 does not is negligible and therefore H_7 and H_8 are computationally indistinguishable.

The proof is completed by noting that, in Hybrid H_8 , the simulator no longer depends on any hidden information of the functionality, and honest parties output the correct output or agree on a corrupt party to blame, as per the definition of security with identifiable abort (Definition 7). Therefore, Hybrid H_8 is identically distributed as the ideal-world experiment.